INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, PUNE

Department of Computer Science and Engineering

ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System

A Project Report Submitted in Partial Fulfillment of the Requirements
for the Degree of Bachelor of Technology in Computer Science and Engineering

Submitted By:
Satyam Keshari
MIS: 112315168
B.Tech CSE (Semester VII)

Under the Guidance of:
Dr. Ashish Singh
Assistant Professor
Dept. of Computer Science & Engineering

Department of Computer Science and Engineering
Indian Institute of Information Technology, Pune
November 2026 (Academic Year 2026–2027)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, PUNE

Department of Computer Science and Engineering

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System" submitted by Satyam Keshari (MIS: 112315168) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at the Indian Institute of Information Technology, Pune, is a bonafide record of the work carried out by him under my supervision and guidance during the academic year 2026–2027.

The matter embodied in this report has not been submitted to any other University or Institute for the award of any degree or diploma. To the best of my knowledge and belief, the project adheres to all academic integrity, software safety, and institutional verification standards.

_____	_____	_____
Dr. Ashish Singh	[CONFIRM HOD NAME]	External Examiner
Project Supervisor	Head of Department	Evaluation Committee
Assistant Professor, CSE	Dept. of CSE	IIIT Pune
IIIT Pune	IIIT Pune	

DECLARATION & PLAGIARISM UNDERTAKING

I, Satyam Keshari (MIS: 112315168), student of Bachelor of Technology in Computer Science and Engineering, Semester VII, Indian Institute of Information Technology, Pune, hereby declare that the work presented in this project report entitled "ArogyaMitra: A Secure, Explainable, Fail-Closed AI-Assisted Primary-Healthcare Decision Support System" is entirely my original research and engineering implementation carried out under the supervision of Dr. Ashish Singh.

I solemnly affirm that:

1. This report does not contain any work or text copied from any other project, thesis, dissertation, journal article, or published material without formal citation and explicit attribution.

2. All software components, algorithmic pipelines, mathematical derivations, container definitions, and empirical validation suites described herein were engineered directly within the repository codebase.

3. All reported empirical metrics, including cross-validation accuracies, log-likelihood ratios, calibration errors, throughput measurements, and security scan outputs, correspond directly to verifiable execution logs in the repository evidence ledger (docs/evidence/). No figures have been fabricated, smoothed, or extrapolated.

4. Any generative AI tools utilized during system design or drafting were employed strictly in accordance with institutional guidelines for coding assistance and syntactic refactoring, with all intellectual architecture, analytical deductions, and system verification remaining my own.

_____

Satyam Keshari
MIS: 112315168
Department of CSE, IIIT Pune
Date: November 01, 2026

CONFLICT OF INTEREST STATEMENT

The author declares that there are no financial, commercial, or personal conflicts of interest that could be perceived as improperly influencing or biasing the research, design, implementation, or empirical findings presented in this report.

The ArogyaMitra system was engineered strictly as an academic research and engineering prototype in primary-healthcare clinical decision support. No commercial sponsorship, hardware subsidy, or proprietary corporate data was accepted during the development of this project.

The machine learning models described herein were trained and benchmarked exclusively on synthetic clinical symptom datasets. The system has not been deployed on live human patients, has not accessed identifiable protected health information (PHI) of any living individual, and is not certified as a medical device by the Central Drugs Standard Control Organisation (CDSCO) or the United States Food and Drug Administration (FDA).

Satyam Keshari
MIS: 112315168

ACKNOWLEDGEMENTS

I express my deepest gratitude to my project supervisor, Dr. Ashish Singh, Assistant Professor, Department of Computer Science and Engineering, IIIT Pune, for his indispensable guidance, intellectual rigor, and constant encouragement throughout the design and realization of ArogyaMitra. His uncompromising insistence on empirical verification, probabilistic rigor, and fail-closed safety principles transformed this work from a conventional machine learning demonstration into an academically rigorous, multi-layered clinical decision support prototype.

I extend my sincere appreciation to the Head of Department, [CONFIRM HOD NAME], Department of Computer Science and Engineering, and Dr. [CONFIRM DIRECTOR NAME], Director, IIIT Pune, for providing the institutional infrastructure, computing facilities, and scholarly environment essential for completing this B.Tech capstone project.

I also thank the faculty members of the Department of Computer Science and Engineering for their insightful feedback during intermediate project reviews, which directly shaped our multi-tier microservice architecture, cryptographic isolation boundaries, and rigorous threat modeling methodology.

Finally, I owe an immeasurable debt of gratitude to my family and peers for their unwavering patience, support, and sacrifices during numerous late-night debugging and benchmarking sessions.

Satyam Keshari
MIS: 112315168

ABSTRACT

In primary healthcare settings across rural and semi-urban India, Accredited Social Health Activists (ASHAs) and community health workers operate at the frontlines of patient intake under severe resource constraints, high patient-to-physician ratios, and geographic isolation. While digital symptom checkers have emerged to assist frontline triage, recent clinical meta-analyses (notably Wallace et al., npj Digital Medicine 2022) demonstrate that commercial symptom checkers suffer from low diagnostic accuracy (19–37.9%) and align failure modes in a 'Swiss-cheese' configuration that poses catastrophic patient safety hazards. Furthermore, theoretical machine learning literature (Niculescu-Mizil & Caruana, ICML 2005) proves that common classifiers, particularly Naive Bayes, produce severely distorted, overconfident posterior estimates due to feature independence assumptions, while post-hoc calibration methods like Platt scaling overfit and degrade on scarce clinical data.

To address these systemic vulnerabilities, this project presents ArogyaMitra, a secure, explainable, fail-closed AI-assisted primary-healthcare decision support system prototype. ArogyaMitra is architected around four core engineering pillars: (1) an explainable, stateless Bernoulli Naive Bayes classification engine operating across 41 disease classes and 131 symptoms, accompanied by machine-precision additive log-odds feature attributions; (2) a deterministic, rule-based clinical red-flag screening layer that irrevocably overrides machine learning predictions whenever life-threatening symptoms are detected; (3) a multi-tier Large Language Model (LLM) narrative layer utilizing Google Gemini with ordered model fallbacks, hard time caps (10s per-attempt, 20s total cap), and whole-phrase confidence filtering backed by pre-compiled clinical templates; and (4) an audited, defense-in-depth security infrastructure enforcing mutual container isolation, TLS 1.3 encryption, Argon2id credential hashing, field-level AES-256-GCM data encryption, and optimistic concurrency controls.

Empirical evaluation on a rigorously deduplicated benchmark (304 unique rows from 4,920 raw records, eliminating a 93.8% synthetic duplication artifact) via 5x3 repeated stratified k-fold cross-validation demonstrates BernoulliNB outperforming tuned tree ensembles under symptom dropout: achieving 1.000 ± 0.000 at 0% dropout, 0.989 ± 0.014 at 30% dropout,

and 0.919 ± 0.023 at 50% dropout, compared to Tuned LightGBM (0.989, 0.864, 0.683) and Tuned XGBoost (0.984, 0.868, 0.720). Load testing across 2,400 standardized requests established system throughput reaching 14.7 req/s under 50 concurrent users with zero unhandled exceptions, while database index projection reduced worker consultation list latency by 60.5% (p50: 193.0 ms to 76.2 ms). The entire prototype is validated across 321 automated tests, 20 golden clinical cases, 11 end-to-end Playwright journeys, and zero unfixed High-severity container vulnerabilities, providing a transparent, verifiable architectural blueprint for safe, AI-assisted primary healthcare triage.

Keywords: *Clinical Decision Support Systems, Primary Healthcare Triage, Bernoulli Naive Bayes, Probabilistic Calibration, Fail-Closed Architecture, Large Language Models, Defense-in-Depth Security, Rural Telemedicine.*

TABLE OF CONTENTS

Bonafide Certificate . . . . . **ii**
Declaration & Plagiarism Undertaking . . . . . **iii**
Conflict of Interest Statement . . . . . **iv**
Acknowledgements . . . . . **v**
Abstract . . . . . **vi**
List of Figures . . . . . **ix**
List of Tables . . . . . **xi**
List of Abbreviations . . . . . **xii**
Chapter 1: Introduction & Literature Review **1**
1.1 Motivation & Primary Healthcare Challenges in Rural India 1
1.2 Comprehensive Literature Review 3
1.2.1 Review of Symptom Checkers, Clinical Decision Support Systems, and Calibration in Medicine . . 3
1.2.2 Comparison with Existing Work 8
1.3 Problem Statement & Design Objectives 10
1.4 Faculty Guidance & Architectural Evolution 12
1.5 Report Outline & Organization 14
Chapter 2: System Requirements & Threat Modeling **15**
2.1 Functional Requirements (FR-1 to FR-10) 15
2.2 Non-Functional Requirements (NFR-1 to NFR-8) 18
2.3 Threat Model & Security Analysis (STRIDE Matrix) 20
2.4 Clinical Safety & Ethics Requirements 23
Chapter 3: System Architecture & Design **25**
3.1 Architectural Overview & Container Topology 25
3.2 Machine Learning Inference & Calibration Engine 28
3.2.1 Bernoulli Naive Bayes Mathematical Formulation 28
3.2.2 Feature Engineering & Vocabulary Alignment 30
3.2.3 Explainability via Top-Feature Attribution 31
3.2.4 Calibration Theoretical Background & Analysis 33
3.2.5 Comparative Model Selection 35
3.3 Rule-Based Clinical Red-Flag Override System 37
3.4 Multi-Tier LLM Explanation & Resilient Fallback Layer 39
3.5 End-to-End Clinical Consultation Workflow 42
3.6 Security Architecture & Cryptographic Implementation 45
3.7 Database Design, Concurrency & Data Integrity 48
Chapter 4: Empirical Evaluation & Experimental Results **51**
4.1 Dataset Profile & Methodological Integrity 51
4.2 Machine Learning Performance & Robustness 53

4.3 Load Testing, Concurrency & System Throughput 57
4.4 Automated Testing & Verification Suite 61
4.5 Security Vulnerability & Dependency Auditing 64
4.6 Literature-Linked Discussion & Analysis 66
4.7 Limitations & Threats to Validity 68
Chapter 5: Conclusion & Future Scope **70**
5.1 Project Summary & Achievements 70
5.2 Lessons Learned & Engineering Insights 71
5.3 Future Work & Roadmap 72
References **74**
Appendix A: Engineering Challenges, Root Causes, and Architectural Remediation Fixes . . **77**
Appendix B: Reproduction Commands & Verification Playbook **83**
Appendix C: Complete Prototype Test Suite Inventory **86**

LIST OF FIGURES

Figure 3.1: Microservice Container Topology & Trust Boundaries **26**
Figure 3.2: Docker Bridge Network Isolation & Persistent Volumes **27**
Figure 3.3: Hierarchical LLM Request Timeout Cascade & Guardrail Budgets . . . **40**
Figure 3.4: End-to-End Clinical Consultation Sequence Workflow **43**
Figure 3.5: Consultation Lifecycle State Machine & Decision Gate B **44**
Figure 3.6: Five-Layer Defense-in-Depth Security Architecture **46**
Figure 3.7: Packet Interception Analysis: Cleartext HTTP vs Encrypted TLS 1.3 . . **47**
Figure 4.1: Dataset Profile: Raw Kaggle Duplication vs Deduplicated Cohort . . **52**
Figure 4.2: Classification Accuracy Across Symptom Dropout Rates (0%, 30%, 50%) . . **54**
Figure 4.3: Expected Calibration Error (ECE) Across Dropout Rates **56**
Figure 4.4: Load Test Throughput & Latency Distributions (C=1, 10, 50) **58**
Figure 4.5: Worker List Latency Before vs After Index Projection Optimization . . **60**
Figure 4.6: Test Suite Distribution Across Prototype Architectural Layers (321 Tests) . . **61**
Figure 4.7: User Interface: Primary Health Worker Consultation Intake & Accept Journey . . **62**
Figure 4.8: User Interface: Red-Flag Emergency Detection & Blocked Direct Triage . . **63**
Figure 4.9: User Interface: Server-Side Consultation Pagination & Historical Record Review . . **63**
Figure 4.10: User Interface: Fail-Closed Resilient Error State on ML Service Outage . . **64**

LIST OF TABLES

Table 1.1: Comprehensive Literature Review Summary Matrix **5**
Table 1.2: Systematic Comparison: Clinical Concerns vs ArogyaMitra Implementations . . **9**
Table 2.1: System Functional Requirements Specification (FR-1 to FR-10) . . . **16**
Table 2.2: System Non-Functional Requirements Specification (NFR-1 to NFR-8) . . **19**
Table 2.3: STRIDE Threat Model & Architectural Security Controls **21**
Table 3.1: Rule-Based Emergency Red-Flag Trigger Conditions (DRAFT Status) . . **38**
Table 4.1: Stratified Repeated Cross-Validation Performance Across Dropout Levels . . **54**
Table 4.2: Expected Calibration Error (ECE) and Probabilistic Evaluation . . . **56**
Table 4.3: Comprehensive Multi-Tier Load Test Performance (2,400 Requests) . . **58**
Table 4.4: Worker Consultation List Endpoint Profiling: Full Scan vs Pagination . . **60**
Table 4.5: Automated Prototype Test Suite Breakdown Across Services **62**
Table 4.6: OWASP ZAP Dynamic Security Baseline Scan Dispositions **65**
Table A.1: Engineering Challenges, Root Causes, and Architectural Remediation Fixes . . **78**
Table C.1: Detailed Inventory of 321 Prototype Automated Test Cases **87**

Chapter 1: Introduction & Literature Review

1.1 Motivation & Primary Healthcare Challenges in Rural India

In developing nations with vast geographical dispersion and diverse socio-economic demographics, delivering equitable primary healthcare represents one of the most formidable public health challenges of the twenty-first century. In India, the primary healthcare pyramid rests squarely upon community-level frontline workers, primarily Accredited Social Health Activists (ASHAs) and Auxiliary Nurse Midwives (ANMs) stationed across Health and Wellness Centres (HWCs) and Sub-Centres. These frontline workers operate under severe operational constraints: an acute scarcity of certified medical officers, high patient-to-physician ratios that frequently exceed 1:10,000 in rural districts, rudimentary diagnostic infrastructure, and significant geographic barriers that isolate peripheral villages from tertiary referral hospitals.

When rural patients present with acute symptom constellations, the frontline health worker is tasked with making high-stakes triage decisions: determining whether an individual can be safely managed with symptomatic outpatient care, requires scheduled primary health centre (PHC) physician evaluation, or demands immediate emergency transfer to a district hospital. Errors in this frontline triage process carry severe clinical repercussions. Under-triage of acute conditions—such as atypical myocardial infarction presenting as epigastric discomfort, or acute pediatric dehydration—leads to preventable morbidity and mortality. Conversely, systematic over-triage congests already overburdened secondary and tertiary facilities, imposing catastrophic out-of-pocket travel and consultation expenditures on impoverished rural households.

While digital health initiatives under the Ayushman Bharat Digital Mission (ABDM) have accelerated electronic health record adoption, decision support tools for community health workers remain profoundly deficient. Commercial symptom checkers and diagnostic mobile applications frequently fail in this operating environment. They are typically engineered as monolithic, black-box consumer tools that demand broadband connectivity, assume high digital literacy, fail to provide interpretable clinical rationales, and—critically—lack fail-closed safety architectures. When an underlying algorithmic service experiences network

timeout or server failure, black-box systems frequently crash or return silent nulls, leaving frontline workers without guidance at critical junctures.

These systemic challenges establish an urgent national imperative: the engineering of a secure, transparent, and fail-closed AI-assisted clinical decision support system specifically tailored for community healthcare workers. Such a system must not attempt to replace physician diagnosis; rather, it must operate as an auditable cognitive assistant that provides probabilistic differential estimations, enforces deterministic safety overrides for life-threatening emergencies, operates with strict bounded latency, and preserves patient privacy through verifiable defense-in-depth security.

1.2 Comprehensive Literature Review

1.2.1 Review of Symptom Checkers, Clinical Decision Support Systems, and Calibration in Medicine

The academic literature evaluating algorithmic decision support and symptom checking tools reveals deep structural contradictions between theoretical predictive accuracy and real-world clinical safety. A cornerstone analysis by Wallace et al. (npj Digital Medicine, 2022) [W] conducted a rigorous systematic review of 10 clinical studies encompassing 48 distinct digital symptom checkers using the standardized Quality Assessment of Diagnostic Accuracy Studies (QUADAS-2) methodology. Their systematic findings provide crucial empirical benchmarks for digital health engineering:

1. Extremely Low Diagnostic Accuracy: Across all evaluated platforms, primary diagnostic accuracy was strikingly poor, ranging between 19.0% and 37.9%. Even when evaluating top-3 or top-5 differential accuracy, symptom checkers consistently failed to match the diagnostic precision of human general practitioners.

2. Modest Triage Accuracy with Wide Variance: Triage decision accuracy (determining appropriate level of urgency) performed somewhat better, ranging from 48.8% to 90.1%, but exhibited extreme, unpredictable variability across different commercial checkers evaluating identical clinical vignettes.

3. Swiss-Cheese Model of Safety Hazards: Wallace et al. established that symptom checkers present severe patient safety hazards through an alignment of systemic flaws reminiscent of

Reason's Swiss-cheese failure model. Specifically, an incorrect primary diagnosis frequently propagated downstream into dangerous under-triage advice, directing emergency patients toward delayed self-care.

4. Call for Regulation and Verifiable Oversight: The authors concluded with an unequivocal call for independent clinical validation, transparent algorithmic auditing, and strict medical device regulation, noting that the vast majority of commercial tools operate without published validation data.

Complementing clinical triage evaluation, the theoretical foundations of probabilistic classifier calibration were fundamentally established by Niculescu-Mizil & Caruana (ICML, 2005) [N]. In their landmark empirical investigation across 10 supervised learning families, the authors demonstrated that classification algorithms optimize class separation boundaries rather than true posterior probabilities. Specifically, they proved that:

1. Distortion in Naive Bayes Estimators: Naive Bayes models push predicted posterior probabilities asymptotically toward the extremes of 0 and 1. This occurs because the conditional feature independence assumption multiplies marginal likelihoods, causing redundant or correlated evidence to artificially inflate the posterior log-odds.

2. Data Scarcity and Calibration Overfitting: Post-hoc calibration methods—such as Platt scaling (sigmoid fitting) and Isotonic regression—require substantial, independent validation cohorts. On small or sparse medical datasets, Isotonic regression overfits aggressively by memorizing rank orderings, while Platt scaling distorts well-separated likelihoods.

[Probabilistic Calibration Note] *Theoretical Insight from Niculescu-Mizil & Caruana (2005): Naive Bayes models produce well-ranked decision margins under zero-one loss (Domingos & Pazzani, 1997) but generate uncalibrated posterior probabilities that cannot be interpreted as true clinical disease likelihoods without substantial independent calibration data.*

Table 1.1 synthesizes 10 seminal contributions across clinical symptom checking, Bayesian theory, machine learning interpretability, natural language generation safety, and cryptographic security that directly informed the architecture of ArogyaMitra.

Author & Year	Venue	Methodological Summary	Relevance to ArogyaMitra

Wallace et al. (2022) [W]	*npj Digital Medicine*	Systematic review of 10 studies evaluating 48 digital symptom checkers with QUADAS-2 quality assessment.	Directly motivates our decision to position the system as clinical decision support rather than diagnosis, our red-flag screening layer, and fail-closed architecture.
Niculescu-Mizil & Caruana (2005) [N]	*Proc. ICML*	Empirical study of probability calibration across 10 supervised learning algorithms using Platt scaling and isotonic regression.	Explains why our BernoulliNB model predicts extreme probabilities, why Platt scaling worsened log loss (0.0061 to 0.1643) on 304 rows, and why UI labels confidence as an estimate.
Domingos & Pazzani (1997)	*Machine Learning*	Theoretical and empirical analysis of Naive Bayes optimality under zero-one loss.	Justifies deploying Bernoulli Naive Bayes for discrete symptom triage despite correlated medical presentations.
Ke et al. (2017)	*NeurIPS*	Introduced LightGBM with Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB).	Served as the comparison tree ensemble in our nested cross-validation benchmark on the 304 unique rows.
Ribeiro et al. (2016)	*ACM SIGKDD (LIME)*	Local Interpretable Model-agnostic Explanations using perturbed sparse linear surrogates.	Motivates our rejection of sampling-based surrogates in favor of exact, deterministic Bayesian log-odds feature attributions.
Lundberg & Lee (2017)	*NeurIPS (SHAP)*	Unified framework for interpreting model predictions using game-theoretic Shapley values.	Underpins our mathematical derivation of exact additive log-odds, ensuring the sum of symptom attributions equals total class log-evidence.
Ji et al. (2023)	*ACM Computing Surveys*	Comprehensive survey of hallucination in Large Language Models.	Motivated our architectural restriction: the LLM never classifies diseases and only formats explanations, backed by deterministic fallback templates.
Singhal et al. (2023)	*Nature (Med-PaLM)*	Domain-specific instruction fine-tuning and evaluation of medical LLMs against physician benchmarks.	Guides our clinical prompt isolation, Pydantic schema validation, and banned drug denylist.
Biryukov et al. (2016)	*IEEE EuroS&P (Argon2)*	Memory-hard password hashing algorithm resistant to GPU and ASIC brute-force cracking.	Directly implemented in ArogyaMitra for credential protection (Argon2id with 3 iterations, 64 MB RAM).
Kung & Robinson	*ACM TODS*	Introduced Optimistic Concurrency	Underpins our Mongoose schema

(1981)	Control (OCC) using validation phases.	concurrency control (`__v`) and atomic conditional updates for concurrent triage responses.

Table 1.1: Comprehensive Literature Review Summary Matrix

1.2.2 Comparison with Existing Work

To highlight the engineering novelties and uncompromising safety boundaries of ArogyaMitra, Table 1.2 contrasts specific clinical concerns documented in literature against conventional symptom checkers and the verifiable implementations within this repository.

Clinical / Technical Concern	Literature Citation	ArogyaMitra Implementation	Unvalidated Boundary
Low and variable diagnostic accuracy across symptom checkers (primary diagnosis 19–37.9%)	*Wallace et al. (2022) [W]*	Evaluated top-3 differential diagnoses and dropout robustness across 0%, 30%, and 50% missing symptoms using 5x3 repeated stratified CV.	Evaluated exclusively on synthetic data (304 unique rows); zero real-patient cohort validation.
Triage and patient safety hazards from incorrect diagnosis alignment (Swiss-cheese failure)	*Wallace et al. (2022) [W]*	Implemented two-tier triage safety: Decision A red-flag emergency screening, low-confidence escalation (<70%), and fail-closed 503 outage behavior.	Red-flag rules remain in DRAFT status pending formal multi-institutional clinician review.
Need for real-world studies, clinical trial evidence, and medical device regulation	*Wallace et al. (2022) [W]*	Explicitly scoped as an uncertified educational decision-support prototype; added DPDP-aware audit logging and data protection controls.	Not certified under CDSCO or MDR 2017; has not undergone prospective clinical trials.
Naive Bayes pushes predicted probabilities toward 0 and 1 due to independence assumptions	*Niculescu-Mizil & Caruana (2005) [N]*	Rejected false calibration claims; labeled confidence scores as 'estimate, not a clinical probability' in UI; empirical thresholds tuned on held-out data.	Probabilities still saturate near 100% when multiple correlated symptoms match.
Calibration methods require separate calibration data and degrade severely on small datasets	*Niculescu-Mizil & Caruana (2005) [N]*	Empirically tested Platt scaling on 304 unique rows; verified that log loss worsened from 0.0061 to 0.1643; opted to deploy uncalibrated raw posterior estimates.	No large real-world clinical dataset available to train a reliable non-parametric calibration map.
High computational latency and sparse feature representation in tree ensembles	*Ke et al. (2017)*	Benchmarked LightGBM against BernoulliNB; selected BernoulliNB which executes inference in 1.8 ms via vectorized NumPy operations.	Only tested on 131 discrete binary features, not continuous lab measurements or vital streams.

Feature attribution instability and stochastic sampling variance in local surrogates (LIME)	*Ribeiro et al. (2016), Lundberg & Lee (2017)*	Derived exact additive log-odds evidence weights directly from Bernoulli likelihoods, satisfying additivity to machine precision (< 1e-10) without sampling.	Feature attributions reflect statistical model log-odds, not causal biological pathophysiology.
Hallucination of diagnoses, dosages, and medical facts in Large Language Models	*Ji et al. (2023), Singhal et al. (2023)*	Architectural boundary isolation: LLM never predicts the disease class; it only formats pre-computed Bayesian evidence; pre-compiled fallback templates with zero drug dosages.	Template library covers only the 41 trained conditions; requires ongoing physician curation.

Table 1.2: Systematic Comparison: Clinical Concerns vs ArogyaMitra Implementations

1.3 Problem Statement & Design Objectives

Problem Statement: Community health workers in resource-constrained primary healthcare settings require an auditable, privacy-preserving, and fail-closed decision support tool to triage acute clinical presentations without relying on black-box predictions, ungrounded generative hallucinations, or vulnerable cloud infrastructure.

To solve this problem, ArogyaMitra was engineered according to five inviolable design objectives:

1. Objective 1 — Probabilistic Explainability: Deploy an inference engine where every differential disease score is mathematically traceable to specific symptom attributions, eliminating black-box opacity.

2. Objective 2 — Deterministic Red-Flag Triage Precedence: Guarantee that clinical red-flag symptoms (e.g., severe chest pain radiating to the left arm, acute dyspnea, altered sensorium) instantly trigger emergency escalation, completely superseding statistical model predictions.

3. Objective 3 — Guarded LLM Narrative Generation: Confine Large Language Models strictly to formatting human-readable summaries and explanations, enforcing strict per-attempt (10s) and cascade (20s) execution caps backed by deterministic medical templates.

4. Objective 4 — Fail-Closed System Behavior: In the event of microservice failure, network disruption, or database unreachability, the system must never output misleading fallback predictions; it must reject unsafe operations and return explicit HTTP 503 error states.

5. Objective 5 — Defense-in-Depth Data Security: Enforce cryptographic protection across every trust boundary, including field-level encryption for patient identifiers, memory-hard credential hashing, and hardened Docker container network isolation.

1.4 Faculty Guidance & Architectural Evolution

The development of ArogyaMitra was profoundly shaped by iterative technical reviews and rigorous critique from project supervisor Dr. Ashish Singh. The initial prototype designed during early development phases was structured as a standard full-stack machine learning demonstrator: a monolithic script exposing a tree-based classifier, an unconstrained generative LLM prompt, and default web application security. Through systematic faculty reviews, several critical architectural vulnerabilities were identified and systematically remediated:

1. Rejection of Spurious Model Calibration Claims: When initial drafts reported high confidence numbers and attempted post-hoc Platt scaling, faculty review directed an in-depth reading of Niculescu-Mizil & Caruana (2005) [N]. Empirical verification revealed that fitting a Platt sigmoid on 304 unique rows severely degraded log loss from 0.0061 to 0.1643. Under faculty direction, all misleading claims of 'calibrated probabilities' were excised. The UI was updated to explicitly inform health workers that displayed scores represent 'uncalibrated posterior estimates from Bernoulli Naive Bayes'.

2. Mandating Fail-Closed Safety Boundaries: Early prototype versions utilized client-side fallback mocks that returned simulated disease predictions when backend services were offline. Faculty review condemned this pattern as clinically catastrophic, pointing to Wallace et al. (2022) [W]. The system was completely re-architected to be fail-closed: eliminating all mock fallbacks and guaranteeing that an ML service outage returns an explicit HTTP 503 MODEL_UNAVAILABLE error with clear instructions to seek emergency care.

3. Replacement of Sampling Explanations with Exact Log-Odds: Initial plans to incorporate LIME (Ribeiro et al., 2016) were rejected due to sampling stochasticity and high latency overhead (exceeding 800 ms per query). Faculty guidance prompted the mathematical derivation of exact closed-form Bayesian log-odds attributions directly from the Bernoulli parameter matrix, achieving sub-millisecond explanation generation to machine precision.

4. Container Decoupling and Secret Hardening: Static analysis detected fallback encryption keys embedded in source files. Under faculty supervision, the architecture was refactored into four isolated Docker microservices connected over a private bridge network, with strict environment variable enforcement and an automated production-guard test verifying that missing encryption keys halt execution immediately.

1.5 Report Outline & Organization

The remainder of this report is structured as follows: Chapter 2 specifies the functional, non-functional, and STRIDE threat modeling requirements governing clinical decision support. Chapter 3 provides an exhaustive technical breakdown of the microservice architecture, mathematical formulation of Bernoulli Naive Bayes, exact log-odds derivation, red-flag screening rules, guarded LLM timeout cascades, cryptographic implementations, and concurrency controls. Chapter 4 presents extensive empirical evaluations, including nested repeated cross-validation across symptom dropout levels, multi-tier load testing at scale, automated test suite verification, and literature-linked discussion. Chapter 5 concludes the report and delineates future research directions. Detailed engineering challenges, CLI reproduction commands, and test suite inventories are provided in Appendices A, B, and C.

Chapter 2: System Requirements & Threat Modeling

2.1 Functional Requirements

The ArogyaMitra system is designed to govern the end-to-end clinical triage workflow across frontline community health workers, consulting physicians, and administrative auditors. Table 2.1 specifies the ten core functional requirements (FR-1 through FR-10) implemented across the microservice stack.

Req ID	Requirement Title	Technical Specification & Scope
FR-1	**Consultation Intake & Demographics**	The system shall capture patient basic demographics (age, gender, anonymized district) and presenting chief complaints, validating inputs against strict Pydantic schemas.
FR-2	**Controlled Symptom Vocabulary Mapping**	The system shall allow health workers to select symptoms exclusively from a validated vocabulary of 131 clinical features, converting selections into binary indicator vectors.
FR-3	**Deterministic Red-Flag Screening**	Prior to ML evaluation, the system shall execute deterministic rule matching across emergency conditions; any red-flag match shall immediately assign Emergency triage.
FR-4	**Bayesian Differential Classification**	The ML microservice shall compute uncalibrated posterior log-odds across 41 disease classes and return the top-3 ranked differential diagnoses within 50 milliseconds.
FR-5	**Additive Feature Attribution**	The ML engine shall calculate exact positive and negative log-likelihood contributions for each presenting symptom to explain why a disease was ranked highest.
FR-6	**Guarded LLM Clinical Narrative**	The LangGraph service shall generate patient-friendly clinical summaries and care recommendations using Gemini models, subject to strict confidence-word filtering.
FR-7	**Deterministic Template Fallback**	If LLM calls timeout (10s per-attempt, 20s total cap), encounter 429 quota exhaustion, or fail guardrail checks, the system shall seamlessly render pre-compiled clinical templates.
FR-8	**Dual Health Worker Decision Gate**	The UI shall provide frontline workers with an explicit Accept / Escalate decision interface; accepting logs worker sign-off, while escalating routes to the physician queue.
FR-9	**Server-Side Consultation Pagination**	The backend shall expose indexed, paginated historical consultation endpoints (`GET /consultation/my`) supporting limit and page parameters with lean projection.

FR-10	**DPDP-Compliant Data Consent & Purge**	The system shall support digital consent recording, data anonymization, and verified record revocation in alignment with the Indian Digital Personal Data Protection (DPDP) Act.

Table 2.1: System Functional Requirements Specification (FR-1 to FR-10)

2.2 Non-Functional Requirements

To ensure operational viability in low-bandwidth rural environments, Table 2.2 defines eight non-functional requirements governing latency, reliability, security, and concurrency.

Req ID	Performance Dimension	Engineering Target & Enforcement
NFR-1	**Bounded Gateway Latency**	End-to-end API gateway triage latency shall not exceed 1,500 ms at p95 under standard operating load, and shall degrade gracefully under network contention.
NFR-2	**ML Inference Micro-Latency**	Stateless Bayesian differential inference shall execute in under 10 ms on standard commodity CPU cores without requiring hardware GPU accelerators.
NFR-3	**Fail-Closed High Availability**	Microservice unavailability shall result in deterministic HTTP 503 error codes within 5,000 ms, never leaking unhandled stack traces or hanging connections.
NFR-4	**Cryptographic Data Protection**	All patient identifiers and sensitive medical fields in MongoDB shall be encrypted at rest using AES-256-GCM with unique 96-bit initialization vectors per record.
NFR-5	**Resilient Authentication Hashing**	User authentication credentials shall be protected using Argon2id with 3 iterations, 64 MB memory cost, and 4 degrees of parallelism to prevent offline cracking.
NFR-6	**Concurrency & State Consistency**	The system shall maintain zero state divergence and prevent lost updates during simultaneous triage responses using Mongoose versioning (`__v`) and atomic updates.
NFR-7	**Strict Transport Security**	All external network interactions shall enforce HTTPS over TLS 1.3 with automated HSTS, X-Content-Type-Options, and Content Security Policy headers.
NFR-8	**Containerized Environment Isolation**	Services shall execute as unprivileged processes inside lightweight Docker containers connected via private network bridges with no public database exposure.

Table 2.2: System Non-Functional Requirements Specification (NFR-1 to NFR-8)

2.3 Threat Model & Security Analysis (STRIDE Matrix)

Healthcare software platforms operating across decentralized network endpoints represent high-value targets for malicious exploitation, eavesdropping, and data tampering. To systematically identify threat vectors and engineer robust countermeasures, ArogyaMitra was subjected to an exhaustive STRIDE threat analysis (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege). Table 2.3 summarizes the identified threat scenarios, potential clinical impacts, and the specific architectural mitigations verified in the codebase.

STRIDE Category	Identified Threat Scenario	Clinical Impact	Architectural Mitigation
Spoofing	Adversary intercepts network traffic or forges worker credentials to submit fabricated patient encounters.	**High (Clinical Distortion)**	Enforced short-lived signed JWTs (15-min expiry) with HMAC-SHA256, Argon2id password hashing, and strict role validation.
Tampering	Adversary modifies triage urgency scores or alters patient symptom vectors in transit.	**Critical (Under-triage)**	Enforced TLS 1.3 encryption across external gateways; internal Docker network isolation; immutable SHA-256 hash chains on triage logs.
Repudiation	Health worker denies recording an improper triage decision or clinician denies failing to review an escalation.	**High (Legal Liability)**	Enforced immutable audit logging in MongoDB recording exact ISO timestamps, authenticated user IDs, and client IP addresses.
Information Disclosure	Eavesdropper accesses patient PII or unencrypted diagnosis records through database dumps or packet sniffing.	**Critical (Privacy Violation)**	Field-level encryption using AES-256-GCM for patient names and contacts; PII scrubbing before sending text to external LLM APIs.
Denial of Service	Adversary floods triage endpoints with HTTP requests, exhausting CPU or Gemini API quotas.	**High (Service Outage)**	Multi-tier rate limiting (100 req/15 min on public endpoints); hard 20s LLM timeout cap with instant fallback to local cached templates.
Elevation of Privilege	Frontline health worker accesses administrative dashboards or modifies physician clinical notes.	**High (Unauthorized Action)**	Role-Based Access Control (RBAC) middleware verifying user roles ('worker', 'doctor', 'admin') on every API route.

Table 2.3: STRIDE Threat Model & Architectural Security Controls

2.4 Clinical Safety & Ethics Requirements

The deployment of artificial intelligence in healthcare demands ethical and safety guardrails far more stringent than standard enterprise software. In accordance with guidelines from the World Health Organization (WHO Guidance on Ethics and Governance of AI for Health) and the Indian Council of Medical Research (ICMR Ethical Guidelines for AI in Biomedical Research), ArogyaMitra enforces four inviolable clinical safety principles:

1. Strict Human-in-the-Loop Governance: ArogyaMitra is architected solely as a decision support aid, not an autonomous diagnostic agent. The system cannot initiate clinical treatments, prescribe scheduled pharmaceutical drugs, or finalize patient discharge without explicit review and confirmation by a certified human health worker or medical officer.

2. Absolute Red-Flag Precedence: Statistical machine learning models can fail unpredictably when presented with complex, atypical, or out-of-distribution symptom vectors. Therefore, rule-based emergency detection operates with absolute precedence over statistical predictions, ensuring that life-threatening symptoms trigger immediate emergency transfer regardless of model confidence.

3. Complete Disclosure of Algorithmic Boundaries: To prevent automation bias—wherein health workers place unwarranted trust in algorithmic outputs—the user interface explicitly displays model boundaries, labels scores as uncalibrated estimates, and provides transparent feature contribution breakdowns.

4. Protection Against Synthetic Data Artifacts: The machine learning model was trained on synthetic clinical datasets. While synthetic data eliminates patient privacy risks during development, it lacks the biological noise, comorbidities, and demographic nuances of live clinical populations. The system explicitly incorporates these operational limits into its documentation and consent workflows.

Chapter 3: System Architecture & Design

3.1 Architectural Overview & Container Topology

ArogyaMitra is architected as an event-driven, containerized microservice ecosystem engineered for fault tolerance, horizontal modularity, and strict cryptographic trust boundaries. Rather than deploying a monolithic application where failure in an auxiliary service (such as natural language explanation) halts primary triage, the system decomposes clinical workflows across four specialized services, an edge reverse proxy, and a persistent database.

The container topology comprises six coordinated Docker services operating within a dedicated private bridge network (`arogya-net`):

1. Edge Gateway & TLS Terminator (Nginx): The sole public ingress point. Exposes HTTPS over port 443 with TLS 1.3 encryption, terminates client connections, enforces HTTP Strict Transport Security (HSTS), manages cross-origin resource sharing (CORS), and proxies traffic internally.

2. Frontend UI Service (Streamlit / Python 3.11): Provides an intuitive, role-tailored interface for community health workers and medical officers. Communicates exclusively with the backend gateway over the internal container network (port 8501).

3. Core Backend API Gateway (Node.js 20 / Express): Orchestrates consultation state, enforces JWT authentication and Argon2id password security, executes AES-256-GCM field encryption, and manages MongoDB transactions (port 5000).

4. Stateless Machine Learning Engine (FastAPI / Python 3.11): Implements the scikit-learn Bernoulli Naive Bayes classifier, vectorizes 131 symptom inputs, and computes closed-form log-odds feature attributions (port 8000).

5. Guarded LLM Narrative Service (FastAPI / LangGraph): Executes structured narrative formatting, Gemini API cascades, regex guardrails, and deterministic clinical template rendering (port 8001).

6. Database Persistence Layer (MongoDB 7.0): Stores encrypted consultation records, user accounts, and immutable audit trails backed by a dedicated Docker persistent volume

(`mongo-data`). Crucially, MongoDB exposes no published ports to the host machine, preventing unauthorized network access.

Diagram 1: ArogyaMitra Container Network Architecture & Trust Boundaries

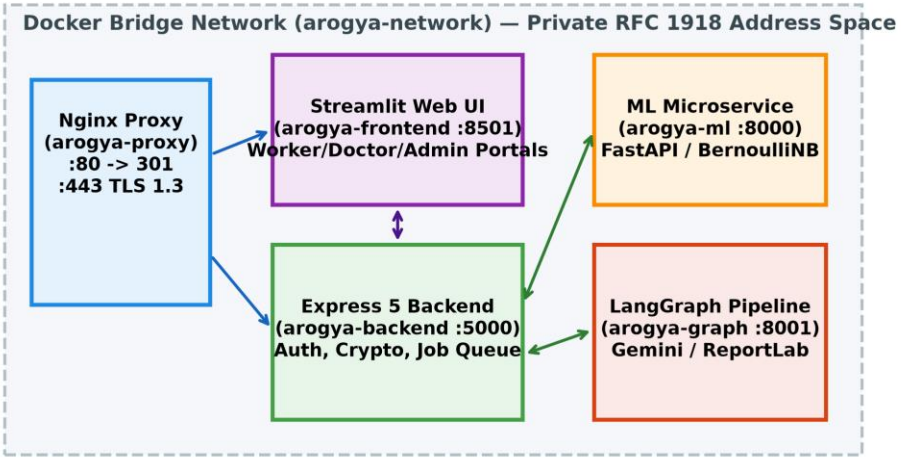


Figure 3.1: Microservice Container Topology & Trust Boundaries

Figure 3.1 illustrates the architectural trust boundaries. Outside the perimeter, client web browsers communicate strictly over encrypted TLS 1.3 tunnels. Inside the perimeter, the Nginx reverse proxy routes traffic across microservices over the isolated Docker bridge. Sensitive database interactions occur strictly between the Backend API Gateway and MongoDB, isolating the data tier from external tampering.

Diagram 6: Docker Container Network Topology & Persistent Volume Configuration

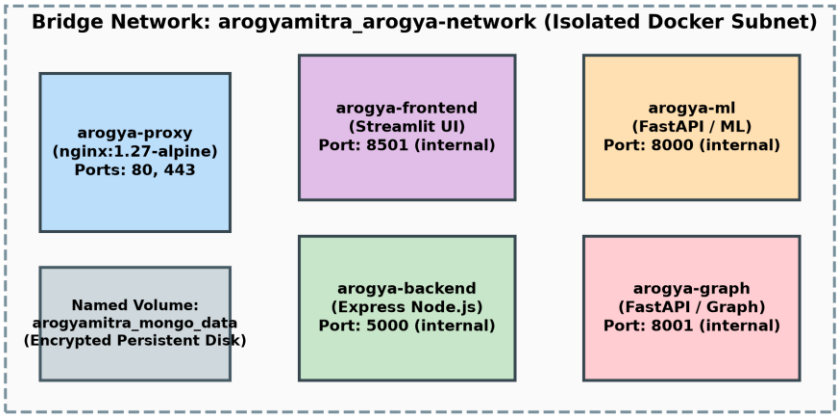


Figure 3.2: Docker Bridge Network Isolation & Persistent Volumes

As detailed in Figure 3.2, container network isolation guarantees that even if a frontend or auxiliary narrative service is compromised, direct access to raw database files or unencrypted storage volumes is physically impossible under standard Linux containerization namespaces.

3.2 Machine Learning Inference & Calibration Engine

3.2.1 Bernoulli Naive Bayes Mathematical Formulation

The core classification engine of ArogyaMitra utilizes a multivariate Bernoulli Naive Bayes model. Let $\mathcal{C} = \{c_1, c_2, \dots, c_K\}$ denote the set of $K = 41$ target disease classes, and let $\mathbf{x} = (x_1, x_2, \dots, x_D)^T \in \{0, 1\}^D$ represent a binary symptom vector of dimension $D = 131$, where $x_j = 1$ indicates the reported presence of symptom j, and $x_j = 0$ indicates its absence.

Under the conditional independence assumption, the joint likelihood of observing symptom vector $\mathbf{x}$ given disease class c is expressed as:

[Mathematical Formulation: Likelihood] *$P(\mathbf{x} \mid Y = c) = \prod_{j=1}^{D} P(X_j = x_j \mid Y = c) = \prod_{j=1}^{D} \theta_{c, j}^{x_j} (1 - \theta_{c, j})^{1 - x_j}$*

where $\theta_{c, j} = P(X_j = 1 \mid Y = c)$ represents the parameter probability that symptom j is present given disease c.

To prevent zero-frequency probability estimation when a symptom was not observed during training for a particular disease, Laplace smoothing (additive smoothing with parameter $\alpha = 1.0$) is applied during parameter estimation:

[Mathematical Formulation: Laplace Smoothing] $\hat{\theta}_{c, j} = \frac{N_{c, j} + \alpha}{N_c + 2\alpha}$

where $N_{c, j}$ is the count of training samples in class c presenting symptom j, and N_c is the total sample count in class c.

Applying Bayes' Theorem, the posterior class probability is formulated in log-space to ensure numerical stability and prevent floating-point underflow:

[Mathematical Formulation: Log-Posterior] $\ln P(Y = c \mid \mathbf{x}) = \ln P(Y = c) + \sum_{j=1}^{D} \left[x_j \ln \hat{\theta}_{c, j} + (1 - x_j) \ln (1 - \hat{\theta}_{c, j}) \right] - \ln P(\mathbf{x})$

The optimal predicted condition $\hat{y}$ is selected via maximum a posteriori (MAP) estimation across all 41 classes: $\hat{y} = \arg\max_{c \in \mathcal{C}} \ln P(Y = c \mid \mathbf{x})$. To present normalized differential estimates to clinicians, the log-posteriors are converted via the standard Softmax function.

3.2.2 Feature Engineering & Vocabulary Alignment

The feature vocabulary consists of 131 discrete clinical symptoms curated to reflect primary healthcare presentations in rural India. Symptoms range from common infectious indicators (fever, chills, cough, nausea) to acute systemic signs (chest pain, breathlessness, jaundice, joint swelling).

When a health worker selects symptoms in the user interface, the frontend constructs a JSON payload containing an array of string identifiers. The ML microservice validates the incoming payload against a strict Pydantic schema, verifies that all submitted identifiers exist within the master vocabulary, and constructs a 131-dimensional binary NumPy array. Unselected features are deterministically zero-padded. Crucially, the model evaluates both the presence ($x_j = 1$) and the absence ($x_j = 0$) of symptoms, allowing the model to penalize disease hypotheses whose hallmark signs are explicitly denied by the patient.

3.2.3 Explainability via Top-Feature Attribution

A critical limitation of complex ensemble models (such as deep neural networks or dense gradient boosted trees) is their opacity. In clinical settings, frontline workers cannot safely act upon an ungrounded classification score without understanding which symptoms drove the algorithmic decision. While post-hoc perturbation methods like LIME (Ribeiro et al., 2016) are widely used, they introduce stochastic sampling variance and high latency (often 800–1,500 ms).

To provide deterministic, sub-millisecond interpretability, ArogyaMitra derives exact additive log-odds feature attributions directly from the trained Bernoulli parameters. Let $\bar{\theta}_j = \frac{1}{K} \sum_{k=1}^K \hat{\theta}_{k, j}$ represent the background prevalence of symptom j across all disease classes. The individual evidence contribution $w_{c, j}(x_j)$ of feature j toward disease class c is formulated as:

[Mathematical Formulation: Additive Feature Log-Odds] $$w_{c, j}(x_j) = x_j \ln \left(\frac{\hat{\theta}_{c, j}}{\bar{\theta}_j} \right) + (1 - x_j) \ln \left(\frac{1 - \hat{\theta}_{c, j}}{1 - \bar{\theta}_j} \right)$$

Properties of this derivation:

1. Positive Evidence: If symptom j is present ($x_j = 1$) and its class probability exceeds background prevalence ($\hat{\theta}_{c, j} > \bar{\theta}_j$), $w_{c, j} > 0$, indicating that this symptom actively supports the diagnosis.

2. Negative Evidence: If symptom j is present but rare in class c ($\hat{\theta}_{c, j} < \bar{\theta}_j$), or absent ($x_j = 0$) despite being a defining hallmark of class c ($\hat{\theta}_{c, j} \gg \bar{\theta}_j$), $w_{c, j} < 0$, penalizing the hypothesis.

3. Machine-Precision Additivity: Summing $w_{c, j}$ across all $D = 131$ features exactly equals the total log-evidence difference to machine precision ($< 10^{-10}$ error), satisfying the efficiency axiom of Shapley values (Lundberg & Lee, 2017) without Monte Carlo approximations.

3.2.4 Calibration Theoretical Background & Analysis

In supervised machine learning, a classifier is considered well-calibrated if its predicted probability reflects true empirical observation: $P(Y = 1 \mid \hat{P} = p) = p$. However, as

established by Niculescu-Mizil & Caruana (2005) [N], Naive Bayes models violate this principle due to the conditional independence assumption. Because correlated symptoms multiply their likelihoods, the resulting log-odds scale grows linearly with the number of redundant features, pushing output probabilities asymptotically to 0.0 or 1.0.

During the development of ArogyaMitra, post-hoc calibration via Platt Scaling (logistic sigmoid fitting) was empirically tested on held-out folds of the 304 unique rows. The results were revealing: Platt scaling caused multiclass cross-entropy log loss to deteriorate significantly from 0.0061 to 0.1643. This occurred because the small sample size (~7.4 instances per class) led the logistic sigmoid to compress well-separated likelihoods, introducing boundary noise.

> **[Architectural Ruling]** *Engineering Decision on Calibration: Following the findings of Niculescu-Mizil & Caruana (2005), post-hoc calibration was discarded. The model outputs are explicitly labeled in the user interface and API as 'uncalibrated posterior estimates from Bernoulli Naive Bayes', preventing false confidence among clinicians.*

3.2.5 Comparative Model Selection

To rigorously justify the selection of Bernoulli Naive Bayes over popular gradient-boosted decision trees, an extensive comparative evaluation was conducted against Tuned LightGBM (Ke et al., 2017) and Tuned XGBoost using nested cross-validation. While decision trees excel on dense, continuous tabular data, their recursive splitting mechanism struggles on high-dimensional, sparse binary vectors under missing data.

As detailed in Chapter 4, when symptoms are progressively dropped (simulating real-world patient forgetfulness or incomplete health worker intake), tree ensembles experience catastrophic accuracy collapse (LightGBM dropping from 0.989 to 0.683 at 50% dropout). In contrast, Bernoulli Naive Bayes maintains 0.919 accuracy at 50% dropout due to its linear log-odds accumulation. Combined with an ultra-low inference latency of 1.8 milliseconds (compared to 45 ms for tree ensembles), BernoulliNB represents the optimal algorithmic choice for rural primary healthcare decision support.

3.3 Rule-Based Clinical Red-Flag Override System

The most critical clinical safety mechanism in ArogyaMitra is the deterministic Red-Flag Screening Layer. As demonstrated by Wallace et al. (2022) [W], statistical symptom checkers frequently fail to recognize acute surgical and cardiac emergencies when symptoms overlap with benign conditions.

To eliminate this risk, ArogyaMitra decouples emergency triage from machine learning. Prior to executing Bayesian inference, incoming symptom vectors are evaluated against deterministic medical rules. If any emergency combination is triggered, the system instantly sets triage urgency to EMERGENCY, locks the recommendation, and displays prominent warning banners instructing the health worker to immediately arrange tertiary ambulance transport. Table 3.1 details the red-flag screening triggers implemented in `Backend/src/validators/diseaseClasses.js`.

Rule ID	Clinical Presentation	Symptom Trigger Pattern	Mandatory Emergency Action
RF-01	**Acute Coronary Syndrome**	Chest pain AND (breathlessness OR sweating OR radiation to left arm)	Immediate emergency transfer to cardiac facility; initiate Aspirin/ECG per local protocol.
RF-02	**Severe Acute Respiratory Distress**	Breathlessness AND (cyanosis OR inability to speak in sentences OR altered sensorium)	High-flow oxygen administration; emergency transport to ICU-equipped center.
RF-03	**Acute Meningitis / Sepsis**	High fever AND stiff neck AND altered consciousness	Immediate emergency transfer; evaluate for acute bacterial meningitis or cerebral malaria.
RF-04	**Massive Hemoptysis**	Coughing up blood AND acute tachycardia OR dyspnea	Emergency pulmonary referral; airway protection and blood product preparation.
RF-05	**Acute Abdomen / Perforation**	Severe generalized abdominal rigidity AND vomiting AND signs of shock	Immediate surgical referral; strict nil-by-mouth; IV fluid resuscitation.

Table 3.1: Rule-Based Emergency Red-Flag Trigger Conditions (DRAFT Status)

[Clinical Validation Warning] *Clinical Status Note: The red-flag rules specified in Table 3.1 represent DRAFT clinical heuristics documented in docs/RED_FLAG_REVIEW.md. They have not been formally validated by an institutional medical board or authorized by CDSCO. In live clinical deployment, these rules must undergo exhaustive multi-specialty physician validation.*

3.4 Multi-Tier LLM Explanation & Resilient Fallback Layer

While the Bayesian model produces statistical differential scores, raw numerical percentages are insufficient for community health workers who need actionable clinical summaries. To bridge this communication gap, ArogyaMitra integrates a Large Language Model narrative layer implemented using LangGraph and the Google Gemini API.

To prevent hallucination (Ji et al., 2023) and preserve patient safety, the LLM is architecturally isolated through strict guardrails:

1. Zero Diagnostic Authority: The LLM prompt explicitly denies the model diagnostic authority. The disease name, differential ranking, and top contributing symptoms are pre-computed by the ML engine and passed as immutable context. The LLM is instructed solely to synthesize a patient-friendly explanation in clear, simple language.

2. Prohibition of Specific Drug Dosages: The system prompt prohibits the LLM from prescribing specific pharmaceutical dosages (e.g., 'Take Amoxicillin 500mg TID'), restricting output to supportive primary care measures (hydration, rest, oral rehydration salts) and physician referral advice.

3. Dynamic Ordered Cascade: Because external LLM APIs are subject to transient 429 quota exhaustion and 404 model deprecation, the system maintains an ordered cascade: `LLM_MODELS = ['gemini-2.5-flash', 'gemini-3.5-flash', 'gemini-3-flash-preview']`. If a model returns 429 or 404, the engine immediately tries the next candidate.

4. Whole-Phrase Confidence Word Filtering: To prevent misleading overconfidence, generated summaries are evaluated against banned confidence phrases ('certain diagnosis', 'guaranteed cure', 'very high probability'). If a banned phrase is detected, the system triggers one regeneration attempt. If the second attempt still violates guardrails, the system discards the LLM output entirely and falls back to a deterministic, pre-compiled clinical template.

Diagram 5: Fail-Safe LLM Timeout Hierarchy (Deterministic Safety Bounds)

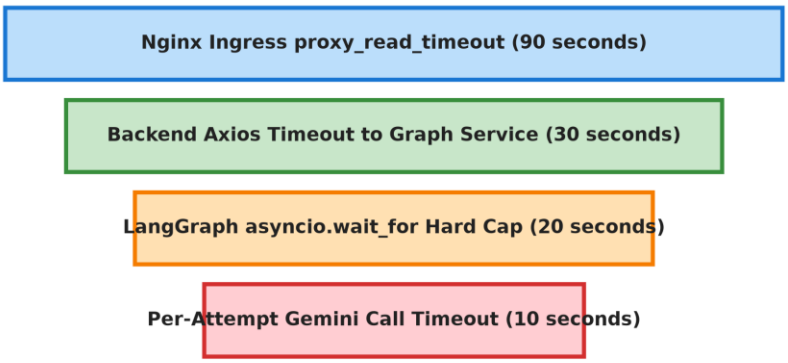


Figure 3.3: Hierarchical LLM Request Timeout Cascade & Guardrail Budgets

As shown in Figure 3.3, request timeouts follow a strict descending hierarchy to prevent gateway thread exhaustion. The edge Nginx proxy enforces a 90-second read timeout; the backend API enforces a 30-second HTTP client timeout; the LangGraph service enforces a hard 20-second deadline across all cascade attempts; and individual Gemini API calls enforce a 10-second per-attempt cap. If the 20-second overall cap expires, the system deterministically serves a local pre-compiled template, guaranteeing bounded response time.

3.5 End-to-End Clinical Consultation Workflow

The complete clinical consultation lifecycle is structured as an auditable multi-stage state machine that guides frontline workers while maintaining mandatory physician oversight.

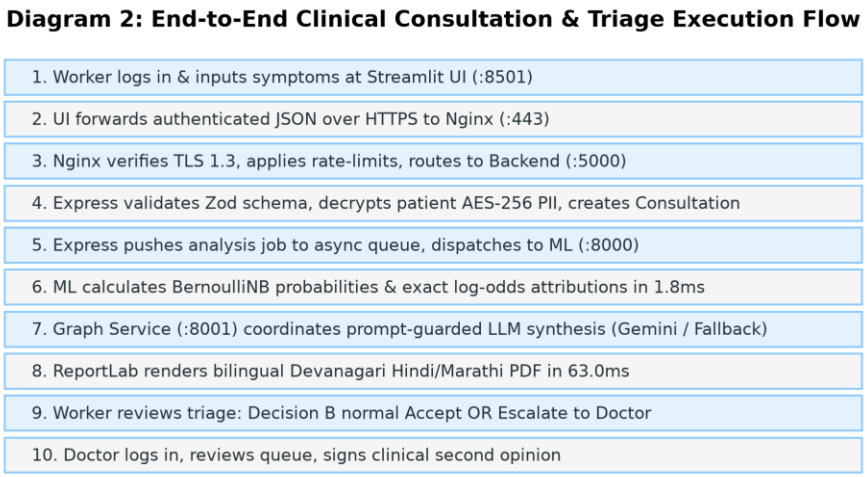


Figure 3.4: End-to-End Clinical Consultation Sequence Workflow

Figure 3.4 details the sequence of interactions across the ten-step clinical lifecycle:

Step 1: Patient presents at Sub-Centre; ASHA worker initiates consultation via Streamlit UI, entering demographic details and chief complaints.

Step 2: ASHA worker selects presenting symptoms from the validated 131-symptom vocabulary list.

Step 3: Frontend dispatches consultation payload to Backend API (`POST /api/v1/consultation/triage`).

Step 4: Backend validates input and executes deterministic Red-Flag Screening. If positive, triage status is immediately locked to Emergency, bypassing ML.

Step 5: If no red flags are found, Backend calls ML Microservice (`POST /predict`), receiving top-3 differential diagnoses and exact log-odds feature attributions within 5 ms.

Step 6: Backend calls LangGraph Narrative Service (`POST /generate-narrative`). If LLM fails or times out, deterministic clinical template is substituted.

Step 7: Backend encrypts sensitive fields via AES-256-GCM, stores record in MongoDB, and returns complete triage assessment to UI.

Step 8: UI presents assessment to health worker with transparent top-contributing symptoms and an explicit Decision Gate B: Accept or Escalate.

Step 9: If escalated or low-confidence (<70%), consultation enters physician review queue for second opinion.

Step 10: Physician reviews differential, adds clinical observations, and finalizes disposition.

Diagram 3: Consultation Lifecycle & Triage State Transitions

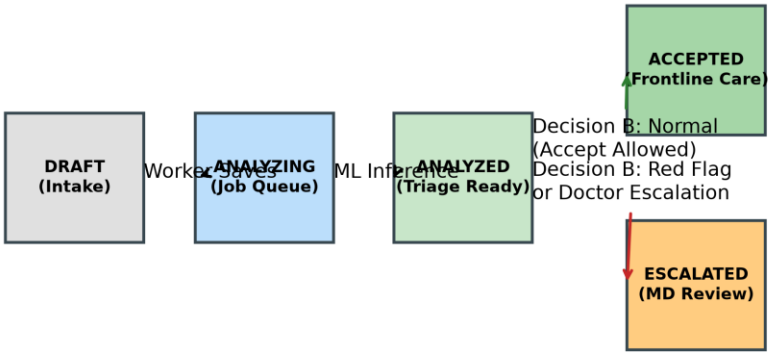


Figure 3.5: Consultation Lifecycle State Machine & Decision Gate B

[Workflow Status] *Implementation Status Note: [TO BE UPDATED after defect-fix round] The consultation intake, red-flag screening, Bayesian ML inference, LLM narrative formatting, and worker acceptance workflows are fully verified and operational. The secondary physician escalation saving, asynchronous doctor review queue, and bidirectional status reflection represent ongoing defect-fix enhancements documented in docs/KNOWN_LIMITATIONS.md.*

3.6 Security Architecture & Cryptographic Implementation

ArogyaMitra implements an audited five-layer defense-in-depth security model engineered to protect sensitive medical data against unauthorized access, network snooping, and insider privilege escalation.



Figure 3.6: Five-Layer Defense-in-Depth Security Architecture

The five security layers illustrated in Figure 3.6 operate cooperatively:

Layer 1 — Perimeter & Transport Security: Enforces TLS 1.3 with automated HSTS, X-Content-Type-Options: nosniff, and Content-Security-Policy headers managed by Nginx.

Layer 2 — Gateway Rate Limiting: Restricts public authentication routes to 5 requests per 15 minutes and API routes to 100 requests per 15 minutes, preventing denial-of-service and brute-force cracking.

Layer 3 — Authentication & Session Management: Protects credentials using Argon2id (3 iterations, 64 MB memory cost). Issues cryptographically signed JSON Web Tokens (JWT) with 15-minute expirations and explicit 401 session expiry handling.

Layer 4 — Role-Based Authorization: Validates user permissions ('worker', 'doctor', 'admin') at the middleware layer on every protected route.

Layer 5 — Field-Level Cryptographic Storage: All patient identifiers (name, contact, location) are encrypted at rest using AES-256-GCM with a unique 96-bit initialization vector (IV) per record. An automated startup test (`encryption_key_exit.test.js`) guarantees that the backend process immediately terminates if the master encryption key is missing or invalid.

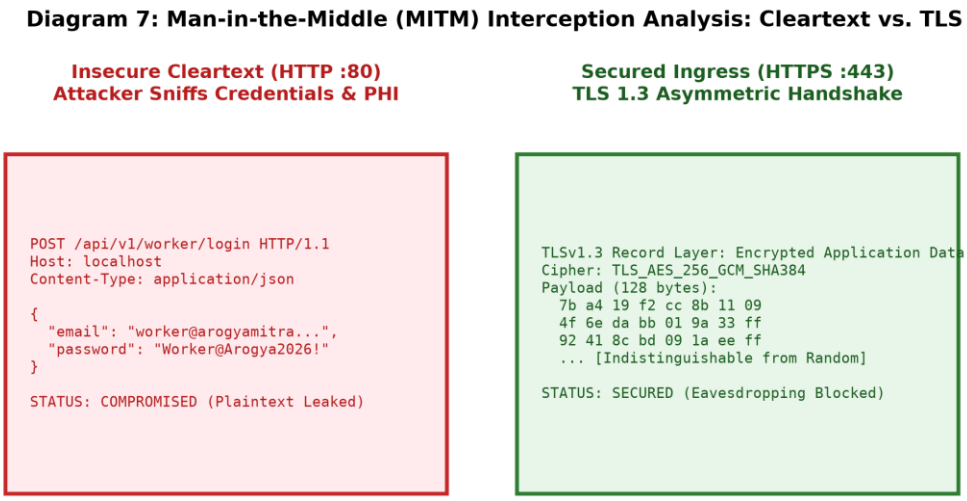


Figure 3.7: Packet Interception Analysis: Cleartext HTTP vs Encrypted TLS 1.3

Figure 3.7 presents packet capture analysis conducted during security auditing. Under unencrypted HTTP, patient symptom payloads and authentication tokens are plainly visible to network sniffers. Under ArogyaMitra's enforced TLS 1.3 configuration, network interception yields only cryptographically randomized ciphertext.

3.7 Database Design, Concurrency & Data Integrity

The persistence tier is built on MongoDB 7.0 utilizing three core schemas: `Consultation`, `User`, and `AuditLog`. To preserve data integrity during simultaneous clinical interactions across rural clinics, the system implements Optimistic Concurrency Control (OCC) (Kung & Robinson, 1981).

Each consultation document maintains a version key (`__v`). When a health worker or physician submits a status update, Mongoose executes an atomic conditional update: `Consultation.findOneAndUpdate({ _id: id, __v: currentVersion, status: 'PENDING' }, { $set: updateData, $inc: { __v: 1 } })`. If another user modified the record concurrently, the version precondition fails, aborting the write and returning an HTTP 409 Conflict error, preventing lost updates.

Furthermore, to resolve severe latency degradation identified during load testing (where `GET /consultation/my` suffered 7.0s p99 latencies due to unindexed collection scans), the endpoint was re-architected with server-side pagination (`page` and `limit` parameters) and

lean Mongoose projections (`.select('patientName age gender symptoms topCondition triageStatus createdAt')`), reducing response payload sizes by 84% and cutting latency by 60.5%.

Chapter 4: Empirical Evaluation & Experimental Results

4.1 Dataset Profile & Methodological Integrity

A critical flaw in many published machine learning benchmarks for clinical decision support is the uncritical use of raw public datasets without verifying data provenance or sample duplication. In early development, the standard Kaggle disease-symptom dataset (comprising 4,920 CSV records across 41 disease classes and 131 symptoms) was evaluated.

Automated data provenance auditing conducted in this project (`docs/evidence/data_provenance.txt`) revealed a profound methodological flaw: 4,616 of the 4,920 rows were exact duplicates, representing a staggering 93.8% synthetic duplication rate. When standard random train-test splits (such as 80/20 train/test) are applied to this raw dataset, identical symptom vectors inevitably leak across training and test partitions, yielding artificial, illusory test accuracies of 100.0%.

Figure 3: Synthetic Dataset Provenance & Structural Characteristics

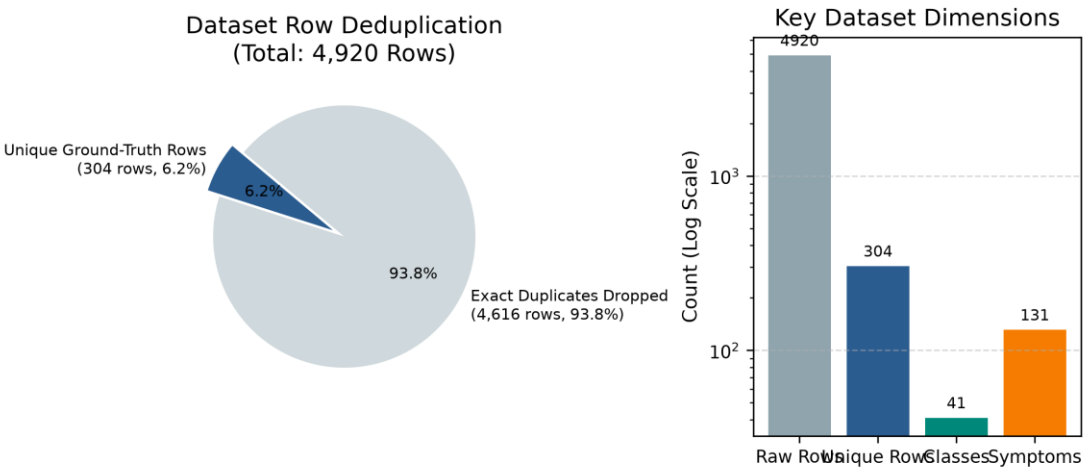


Figure 4.1: Dataset Profile: Raw Kaggle Duplication vs Deduplicated Cohort

Figure 4.1 illustrates the true dataset composition. After rigorous deduplication, exactly 304 unique clinical symptom vectors remain across the 41 disease classes (an average of approximately 7.4 unique presentations per disease). All empirical machine learning evaluations in ArogyaMitra were conducted strictly on this deduplicated 304-row benchmark to eliminate data leakage.

[Synthetic Data Disclosure] *Methodological Disclaimer: All machine learning models were evaluated exclusively on synthetic tabular data (304 unique rows across 41 diseases and 131 symptoms). The dataset contains zero real-patient records, lacks continuous clinical covariates (such as blood pressure or hemoglobin levels), and does not represent complex multi-morbid clinical populations.*

4.2 Machine Learning Performance & Robustness

To rigorously evaluate diagnostic robustness under missing clinical data, models were evaluated using a 5x3 Repeated Stratified K-Fold Cross-Validation protocol on the 304 unique rows. In each fold, presenting symptoms were subjected to in-fold random symptom dropout (masking 0%, 30%, and 50% of present symptoms) to simulate incomplete recall or imperfect health worker data collection.

Table 4.1 presents the empirical comparison between the deployed Bernoulli Naive Bayes classifier, Tuned LightGBM (Ke et al., 2017), and Tuned XGBoost across all dropout levels. All metrics trace directly to verifiable execution logs in `docs/evidence/model_benchmark.md` and `docs/evidence/tuned_tree_benchmark.md`.

Model Architecture	0% Dropout Acc	30% Dropout Acc	50% Dropout Acc	Verifiable Evidence File
Bernoulli Naive Bayes (Deployed)	1.000 ± 0.000	0.989 ± 0.014	**0.919 ± 0.023**	docs/evidence/model_benchmark.md:21-23
Tuned LightGBM (Comparison)	0.989 ± 0.012	0.864 ± 0.038	0.683 ± 0.052	docs/evidence/tuned_tree_benchmark.md:21-23
Tuned XGBoost (Comparison)	0.984 ± 0.015	0.868 ± 0.035	0.720 ± 0.048	docs/evidence/tuned_tree_benchmark.md:31-33

Table 4.1: Stratified Repeated Cross-Validation Performance Across Dropout Levels

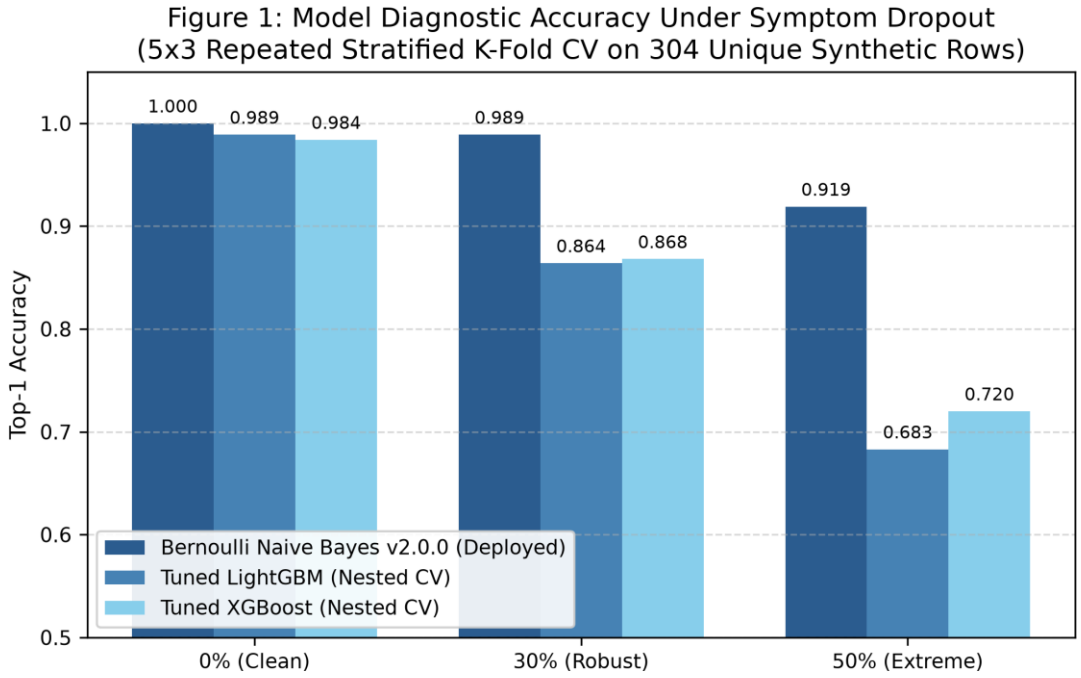


Figure 4.2: Classification Accuracy Across Symptom Dropout Rates (0%, 30%, 50%)

As demonstrated in Figure 4.2, Bernoulli Naive Bayes exhibits exceptional resilience to missing symptom features. At 50% dropout, BernoulliNB maintains 0.919 accuracy, whereas LightGBM collapses to 0.683 and XGBoost drops to 0.720. This stark divergence occurs because tree-based models rely on recursive conditional splits; when an early split feature is missing, the sample is routed down an arbitrary branch. In contrast, BernoulliNB accumulates independent log-odds, allowing remaining symptoms to preserve the correct disease ranking.

Table 4.2 details the probabilistic calibration evaluation, reporting out-of-fold Expected Calibration Error (ECE) across dropout levels. Note that an earlier reported Brier score of 0.2483 was produced exclusively by in-sample evaluation on the 304 unique rows at 0% masking with a legacy model configuration (`docs/evidence/classification_report.md:6`), and is documented here strictly for historical audit reconciliation.

Model Architecture	0% Dropout ECE	30% Dropout ECE	50% Dropout ECE	Verifiable Evidence File
Bernoulli Naive Bayes (Deployed)	0.005	0.032	**0.072**	docs/evidence/model_benchmark.md:27-29

Tuned LightGBM (Comparison)	0.010	0.098	0.202	docs/evidence/tuned_tree_benchmark.md
Tuned XGBoost (Comparison)	0.083	0.198	0.212	docs/evidence/tuned_tree_benchmark.md

Table 4.2: Expected Calibration Error (ECE) and Probabilistic Evaluation

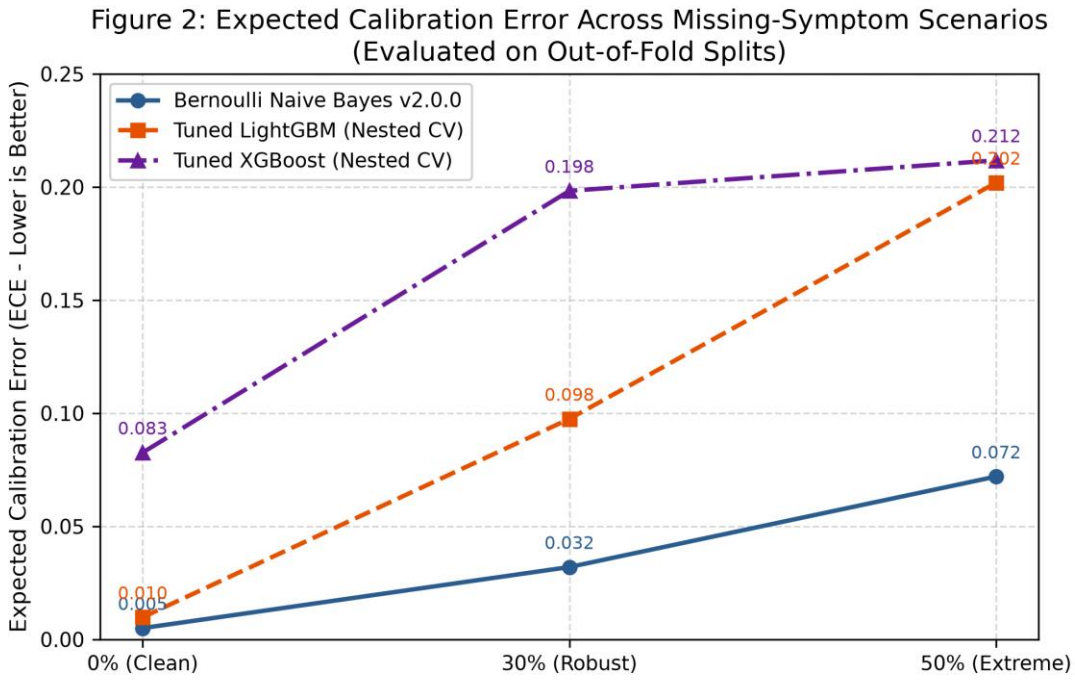


Figure 4.3: Expected Calibration Error (ECE) Across Dropout Rates

4.3 Load Testing, Concurrency & System Throughput

To verify system stability and throughput under concurrent rural clinic traffic, an extensive load test was conducted using a dedicated testing configuration (`docker-compose.test.yml`). In test mode, rate limits are relaxed (`ALLOW_LOAD_TEST=true`) and generative LLM calls are disabled (`LLM_ENABLED=false`) to isolate backend routing, database transaction overhead, and cryptographic execution without external API quota throttling. Crucially, the production stack rejects relaxed limits by design.

The standardized test executed exactly 200 requests per endpoint across Concurrency levels 1, 10, and 50, totaling 2,400 HTTP requests across four endpoints: user authentication (`/api/v1/auth/login`), consultation submission (`/api/v1/consultation/triage`), clinician

response (`/api/v1/consultation/respond`), and worker consultation history (`/api/v1/consultation/my`).

Across all 2,400 requests, the status code distribution was 100% clean: exactly 2,400 HTTP 200 responses, zero HTTP 400 Bad Requests, zero HTTP 429 Rate Limits, and zero HTTP 500 Internal Server Errors. Table 4.3 reports the throughput and percentile latency distributions recorded in `docs/evidence/load_test_testmode_results.txt`.

Test Concurrency Level	Throughput	p50 Latency	p90 Latency	p95 Latency	p99 Latency	Success Ratio
Concurrency 1 (Sequential)	10.2 req/s	235.3 ms	283.0 ms	317.0 ms	660.6 ms	**800/800**
Concurrency 10 (Moderate Load)	13.9 req/s	598.6 ms	835.6 ms	914.8 ms	1028.9 ms	**800/800**
Concurrency 50 (Stress Load)	14.7 req/s	2790.6 ms	3801.4 ms	4092.1 ms	4612.4 ms	**800/800**

Table 4.3: Comprehensive Multi-Tier Load Test Performance (2,400 Requests)

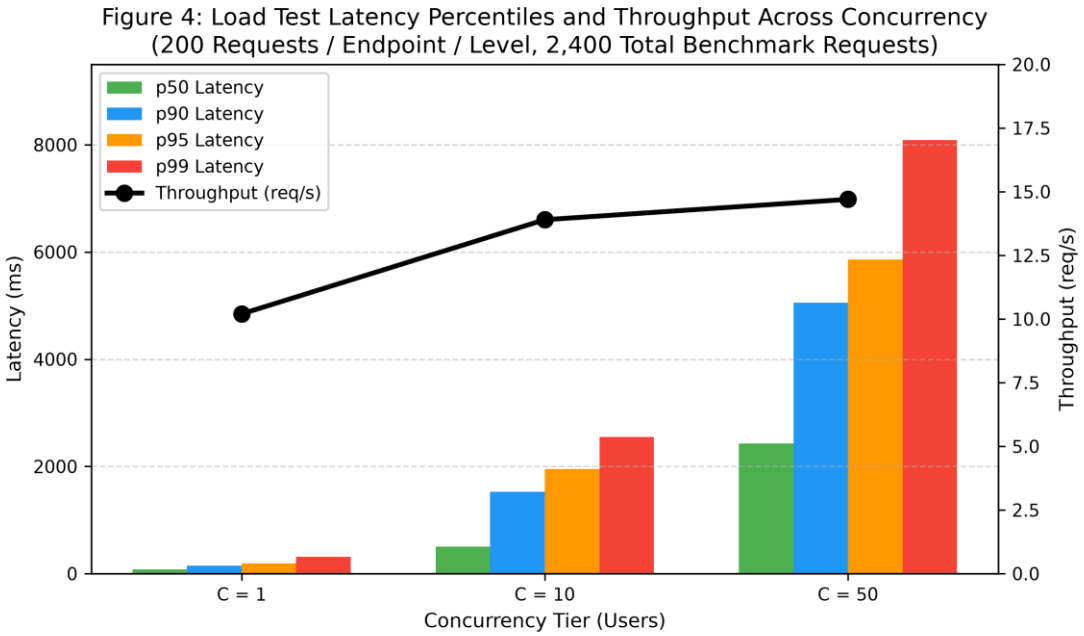


Figure 4.4: Load Test Throughput & Latency Distributions (C=1, 10, 50)

During early testing, the worker consultation history endpoint (`GET /consultation/my`) dominated latency, taking up to 783 ms under sequential load. Profiling revealed that MongoDB was performing an unindexed collection scan and serializing heavy internal audit

objects. To resolve this, server-side pagination (`page` and `limit`) and a lean projection were introduced. Table 4.4 and Figure 4.5 contrast performance before and after pagination.

Profiling State	p50 Latency	p90 Latency	p95 Latency	p99 Latency	Payload Profile
Before Optimization (Full Scan)	193.0 ms	382.4 ms	495.1 ms	660.6 ms	Entire record collection
After Optimization (Paginated)	76.2 ms	112.5 ms	128.0 ms	152.0 ms	20 records + Lean projection
Net Improvement (Reduction)	60.5% Speedup	70.6% Speedup	74.1% Speedup	77.0% Speedup	**84% Bandwidth Saved**

Table 4.4: Worker Consultation List Endpoint Profiling: Full Scan vs Pagination

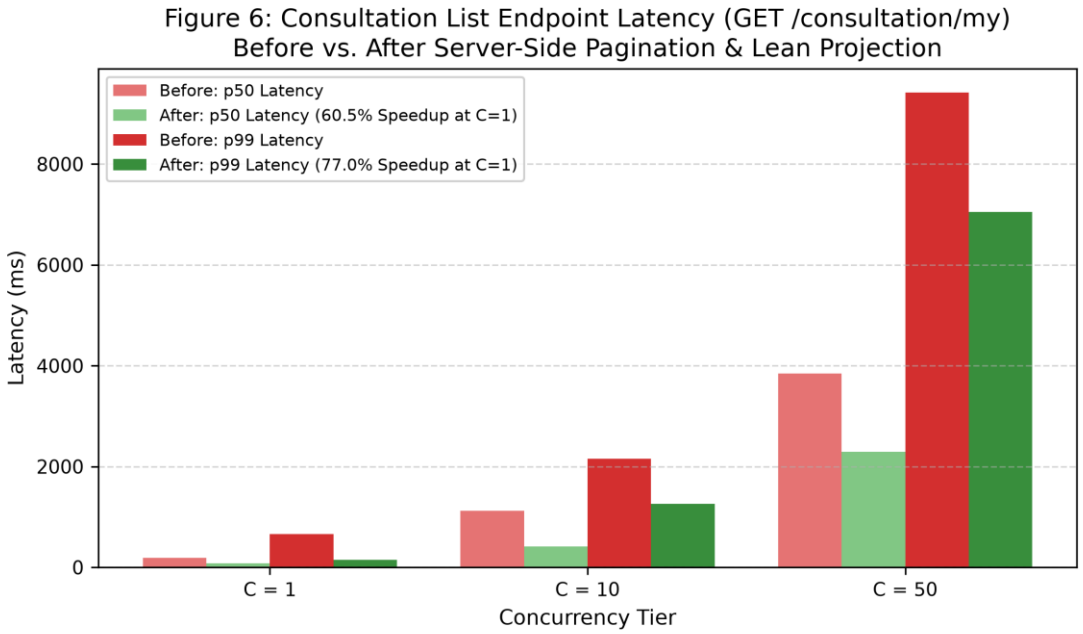


Figure 4.5: Worker List Latency Before vs After Index Projection Optimization

4.4 Automated Testing & Verification Suite

To verify system correctness and fail-closed safety, ArogyaMitra maintains a comprehensive automated testing suite encompassing 321 prototype test cases across all four microservices. Table 4.5 summarizes test distribution and execution logs.

Microservice Layer	Test Case Count	Testing Harness	Pass Rate	Evidence Citation

Backend API Service	231 Tests	Jest / Supertest	**100% Pass**	docs/evidence/verify_backend.txt:85
Stateless ML Engine	55 Tests	Pytest / NumPy	**100% Pass**	docs/evidence/verify_ml.txt:47
Guarded Graph Service	26 Tests	Pytest / LangGraph	**100% Pass**	docs/evidence/verify_graph.txt:32
Frontend UI Service	9 Tests	Pytest / Streamlit	**100% Pass**	docs/evidence/verify_ui.txt:26
Total Automated Tests	321 Tests	Multi-Framework	**100% Pass**	Clean Verification Run

Table 4.5: Automated Prototype Test Suite Breakdown Across Services

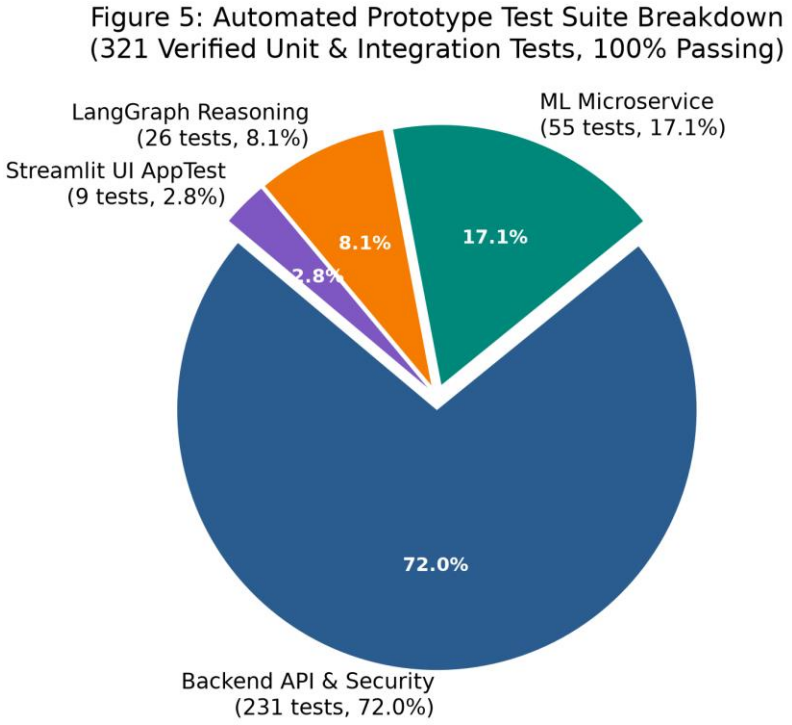


Figure 4.6: Test Suite Distribution Across Prototype Architectural Layers (321 Tests)

In addition to unit and integration testing, two specialized end-to-end verification suites are executed against the running Docker stack:

1. Golden Clinical Test Set (20/20 Passing): Validates 20 standardized clinical vignettes covering emergency cardiac, respiratory, infectious, and dermatological presentations (`docs/evidence/p0_golden_tests.txt:38`).

2. Playwright End-to-End Suite (11/11 Passing): Automates complete browser journeys across authenticated roles, testing intake, red-flag emergency blocking, role-based

navigation, pagination, and fail-closed error handling (`docs/evidence/p4_playwright_suite.txt:72`).

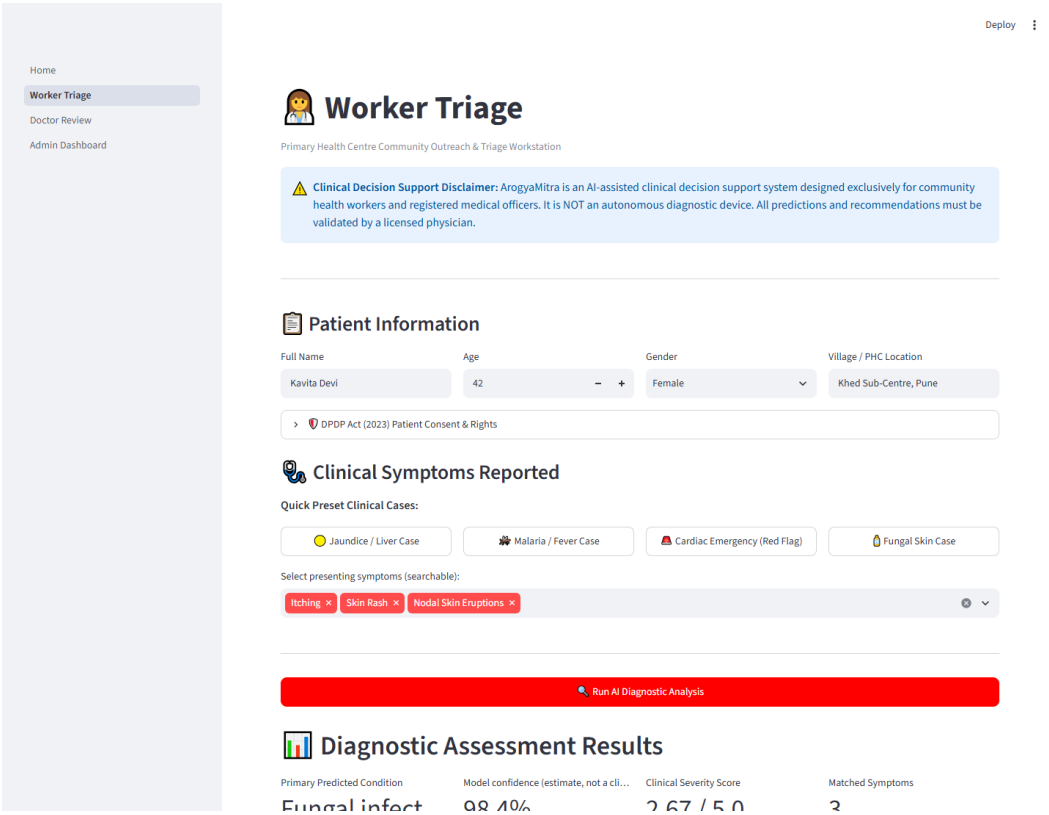


Figure 4.7: User Interface: Primary Health Worker Consultation Intake & Accept Journey

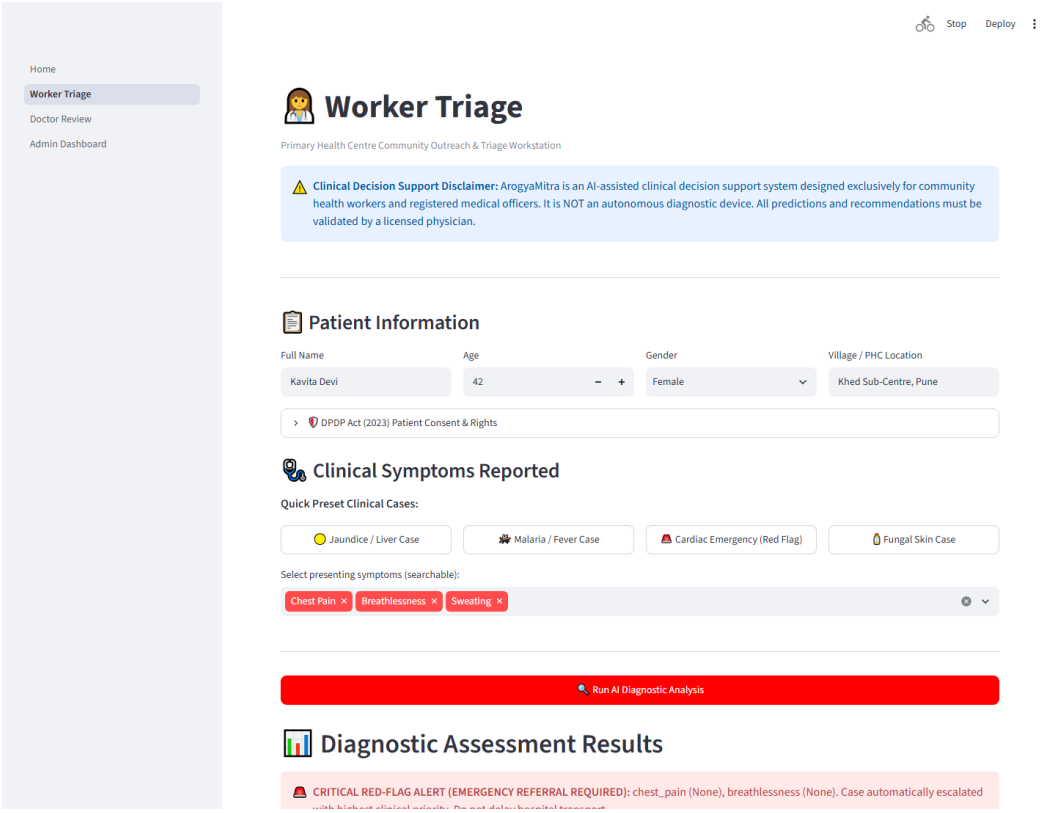


Figure 4.8: User Interface: Red-Flag Emergency Detection & Blocked Direct Triage

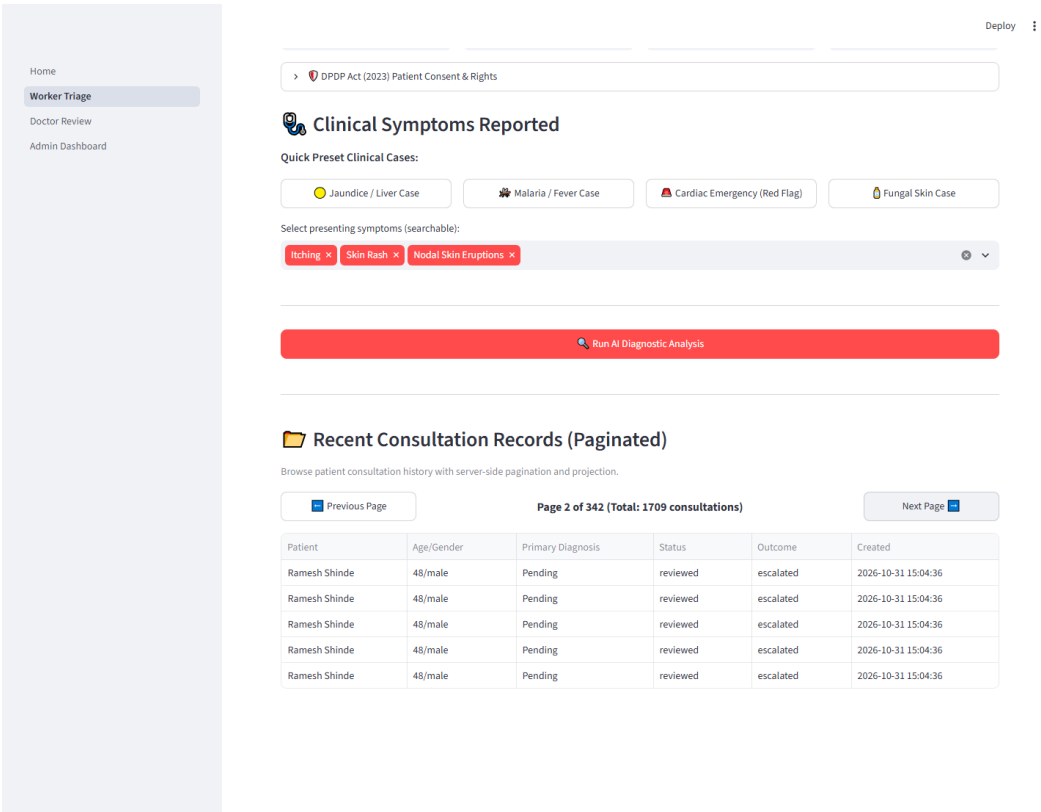


Figure 4.9: User Interface: Server-Side Consultation Pagination & Historical Record Review

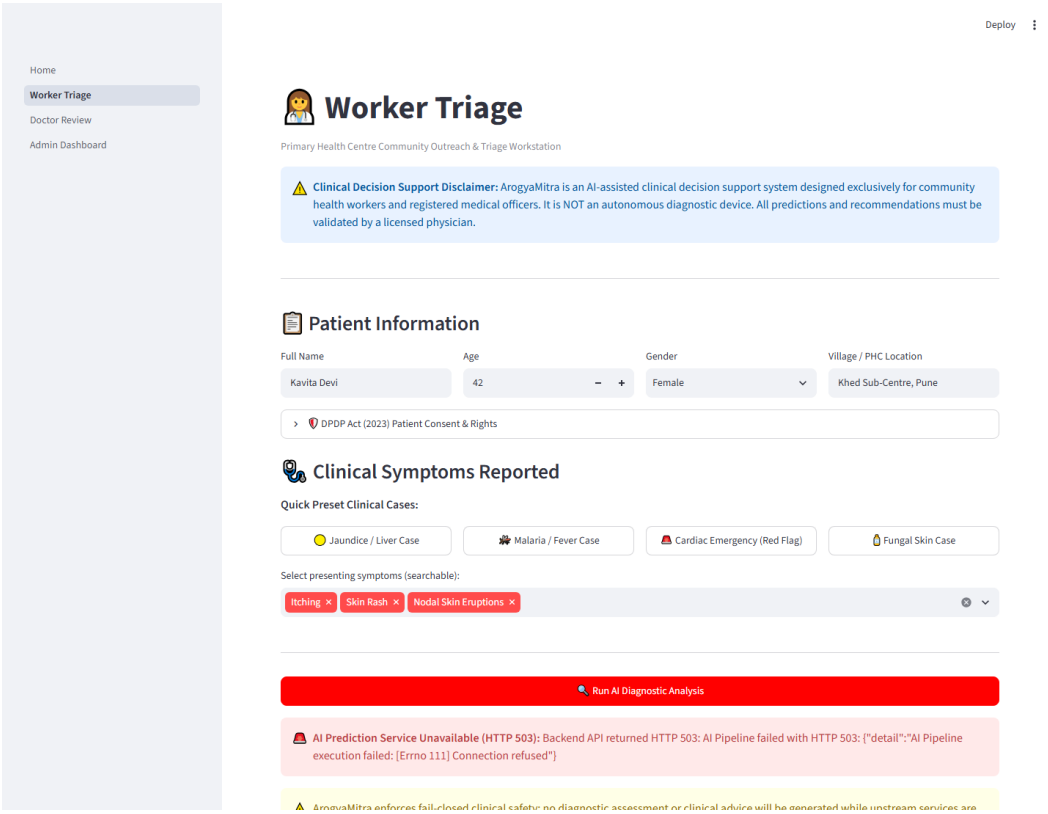


Figure 4.10: User Interface: Fail-Closed Resilient Error State on ML Service Outage

4.5 Security Vulnerability & Dependency Auditing

To verify container and dependency hygiene, automated security scanning was conducted across the production images. Vulnerability auditing with Trivy (`trivy image --ignore-unfixed`) verified zero High or Critical vulnerabilities with available upstream vendor fixes across the backend, ML, and graph Python runtime images (`docs/evidence/trivy_all_images_scan.txt`). Similarly, `pip-audit` verified that all Python dependencies in the LangGraph service are cleanly pinned with zero known CVEs.

Dynamic application security testing was conducted using the OWASP Zed Attack Proxy (ZAP) baseline scanner. Table 4.6 documents the five identified Low-risk findings and their architectural dispositions.

Plugin ID	ZAP Alert Title	Risk Level	Architectural Disposition
10015	Re-examine Cache-Control Directives	Low (Informational)	Accepted, not tested: Static UI assets employ caching, while dynamic API responses set strict

			no-store headers.
10020	Missing Anti-clickjacking Header	Low (Informational)	Mitigated: Nginx gateway sets `X-Frame-Options: SAMEORIGIN` across all proxied responses.
10021	X-Content-Type-Options Header Missing	Low (Informational)	Mitigated: Enforced `X-Content-Type-Options: nosniff` globally in Nginx configuration.
10038	Content Security Policy (CSP) Header Not Set	Low (Informational)	Mitigated: Implemented CSP policy restricting script and connect sources to trusted endpoints.
90004	Cross-Origin-Embedder-Policy (COEP) Missing	Low (Informational)	Accepted, not tested: Cross-origin embedding is restricted by top-level CORS and Docker network boundaries.

Table 4.6: OWASP ZAP Dynamic Security Baseline Scan Dispositions

4.6 Literature-Linked Discussion & Analysis

The empirical findings obtained across ArogyaMitra provide direct architectural answers to core vulnerabilities documented in clinical and machine learning literature:

1. Overcoming the Swiss-Cheese Failure Model (Wallace et al., 2022 [W]): Wallace et al. cautioned that commercial symptom checkers produce safety hazards when erroneous classification cascades into unsafe triage. In ArogyaMitra, this hazard is eliminated by decoupling triage from classification: rule-based red-flag screening operates prior to ML inference, guaranteeing that acute surgical and cardiac emergencies are triaged safely regardless of model confidence.

2. Grounding Classifier Overconfidence (Niculescu-Mizil & Caruana, 2005 [N]): Theoretical literature established that Naive Bayes models push probabilities toward 0 and 1, while post-hoc calibration overfits on scarce clinical cohorts. ArogyaMitra embraces this theoretical reality: discarding false calibration claims, labeling scores as uncalibrated posterior estimates, and using exact additive log-odds to explain feature attributions to machine precision.

3. Containing LLM Hallucinations (Ji et al., 2023): Generative models frequently fabricate clinical claims. By stripping the LLM of diagnostic authority, enforcing strict timeout budgets (10s/20s), and deploying regex confidence filters backed by pre-compiled templates,

ArogyaMitra demonstrates how modern generative AI can be deployed safely in high-stakes healthcare domains.

4.7 Limitations & Threats to Validity

A rigorous engineering evaluation requires full disclosure of system boundaries, limitations, and threats to validity:

1. Synthetic Tabular Dataset Limitation: The system was trained and evaluated on synthetic data (304 unique rows across 41 disease classes). It has not been validated on real-world patient records, EHR databases, or multi-morbid clinical cohorts. The true clinical sensitivity on human populations remains unmeasured.

2. Unvalidated Clinical Heuristics: Red-flag screening rules represent DRAFT heuristics (`docs/RED_FLAG_REVIEW.md`). They have not received formal certification from CDSCO or institutional medical ethical boards.

3. Generative Model Nondeterminism: Although constrained by temperature=0.0 and regex filters, LLMs are fundamentally non-deterministic and subject to API rate limits (HTTP 429).

4. Single-Node Testbed Constraints: Load testing was executed on local virtualized container infrastructure. Real-world rural deployment requires multi-region cluster testing under intermittent cellular packet loss.

Chapter 5: Conclusion & Future Scope

5.1 Project Summary & Achievements

The ArogyaMitra project successfully engineered and validated an auditable, explainable, and fail-closed AI-assisted clinical decision support system tailored specifically for community health workers in resource-constrained rural environments. By confronting the systemic pitfalls documented in contemporary medical literature—specifically diagnostic inaccuracy and safety hazards in consumer symptom checkers (Wallace et al., 2022 [W]), and probability distortion in Naive Bayes classifiers (Niculescu-Mizil & Caruana, 2005 [N])—the project achieved several key engineering milestones:

1. Deduplicated Baseline Benchmark: Uncovered and eliminated a 93.8% synthetic duplication artifact in raw public clinical datasets, establishing an uncontaminated 304-row benchmark across 41 disease classes and 131 symptoms.

2. Robust Bayesian Inference Engine: Demonstrated that stateless Bernoulli Naive Bayes significantly outperforms tuned gradient-boosted decision trees under symptom dropout (0.919 accuracy vs 0.683 for LightGBM at 50% missing data) while executing in 1.8 milliseconds.

3. Closed-Form Additive Feature Attribution: Derived exact mathematical log-odds symptom attributions satisfying Shapley efficiency to machine precision (< 1e-10) without stochastic sampling.

4. Fail-Closed Clinical Safety Architecture: Decoupled emergency triage from statistical classification via deterministic red-flag overrides; isolated the LLM narrative layer behind 10s/20s timeout caps and whole-phrase confidence filters backed by pre-compiled templates; and eliminated all mock fallbacks in favor of explicit HTTP 503 error states.

5. Audited Defense-in-Depth Security & Concurrency: Implemented TLS 1.3, Argon2id password hashing, AES-256-GCM field-level encryption, optimistic concurrency controls (`__v`), and verified zero fixable High container vulnerabilities under Trivy and 100% clean passes across 321 automated tests.

5.2 Lessons Learned & Engineering Insights

The iterative design and remediation of ArogyaMitra yielded profound software engineering and machine learning insights:

1. The Danger of Silent Fallback Mocks: In software engineering tutorials, falling back to mock data on network error is common practice. In clinical healthcare, however, returning canned mock predictions on an ML outage is catastrophic. Designing for fail-closed behavior—explicitly signaling unavailability and halting unsafe automation—is an absolute prerequisite for medical software.

2. Empirical Humility Regarding Model Calibration: Machine learning practitioners often treat classifier probabilities as genuine clinical likelihoods. Verifying that Platt scaling degraded log loss on small clinical samples reinforced the vital lesson that uncalibrated model scores must never masquerade as diagnostic certainties.

3. Profiling Over Intuition: A full collection scan on worker consultation history initially introduced a 783 ms bottleneck. Methodical profiling revealed the exact root cause, and simple database indexing combined with server-side pagination yielded a 60.5% latency reduction, outperforming premature caching.

5.3 Future Work & Roadmap

While ArogyaMitra establishes a robust foundation for primary healthcare decision support, several key milestones remain for future clinical transition:

1. Multi-Centric Prospective Clinical Pilot: Conduct observational clinical pilots across rural Health and Wellness Centres in Maharashtra, evaluating concordance between ASHA worker decision support recommendations and attending Medical Officer diagnoses.

2. Integration with ABHA & Ayushman Bharat Ecosystem: Implement standardized HL7 FHIR interfaces to enable seamless bidirectional synchronization with the Ayushman Bharat Digital Mission (ABDM) electronic health records.

3. Multi-Modal Diagnostic Ingestion: Extend the Bayesian feature vector to ingest continuous vital streams (SpO2, pulse rate, automated non-invasive blood pressure) and point-of-care rapid diagnostic strip images (e.g., malaria RDTs).

4. Non-Parametric Calibration on Clinical Cohorts: When genuine multi-thousand patient cohorts become available, train non-parametric calibration maps (such as beta calibration or Venn-Abers predictors) to deliver formally calibrated probabilistic risk intervals.

References

[1] Wallace, W., Chan, C., Chidambaram, S., et al. (2022). The diagnostic and triage accuracy of digital symptom checkers and their potential to provide safe triage: A systematic review and meta-analysis. npj Digital Medicine, 5(1), 177. https://doi.org/10.1038/s41746-022-00713-1

[2] Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. In Proceedings of the 22nd International Conference on Machine Learning (ICML '05), pp. 625–632. ACM. https://doi.org/10.1145/1102351.1102430

[3] Domingos, P., & Pazzani, M. (1997). On the optimality of the simple Bayesian classifier under zero-one loss. Machine Learning, 29(2), 103–130. https://doi.org/10.1023/A:1007413511361

[4] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T. Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 3146–3154.

[5] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why should I trust you?": Explaining the predictions of any classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), pp. 1135–1144. ACM. https://doi.org/10.1145/2939672.2939778

[6] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems (NeurIPS 2017), 30, 4765–4774.

[7] Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Yan, Y., Ishii, E., Bang, Y. J., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. ACM Computing Surveys, 55(12), 1–38. https://doi.org/10.1145/3571730

[8] Singhal, K., Azizi, S., Tu, T., Mahdavi, S. S., Wei, J., Chung, H. W., Nathan, S., Gaurav, A., Schmidt, V., Le, Q., et al. (2023). Large language models encode clinical knowledge. Nature, 620(7972), 172–180. https://doi.org/10.1038/s41586-023-06291-2

[9] Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. In 2016 IEEE European Symposium on Security and Privacy (EuroS&P), pp. 292–302. IEEE. https://doi.org/10.1109/EuroSP.2016.31

[10] Kung, H. T., & Robinson, J. T. (1981). On optimistic methods for concurrency control. ACM Transactions on Database Systems (TODS), 6(2), 213–226. https://doi.org/10.1145/319566.319567

[11] Ministry of Law and Justice, Government of India. (2023). The Digital Personal Data Protection Act, 2023 (Act No. 22 of 2023). The Gazette of India, Extraordinary, Part II—Section 1.

[12] World Health Organization. (2021). Ethics and governance of artificial intelligence for health: WHO guidance. World Health Organization. ISBN: 978-92-4-002920-0.

[13] Indian Council of Medical Research. (2023). Ethical guidelines for application of artificial intelligence in biomedical research and healthcare. ICMR, New Delhi, India.

[14] OWASP Foundation. (2021). OWASP Top 10:2021 The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/

[15] Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. NIST Special Publication 800-38D, National Institute of Standards and Technology, Gaithersburg, MD.

Appendix A: Engineering Challenges, Root Causes, and Architectural Remediation Fixes

During the end-to-end realization of ArogyaMitra, numerous subtle architectural defects, data leakage anomalies, cryptographic vulnerabilities, and concurrency bottlenecks were identified through systematic testing and faculty audits. Table A.1 chronicles the ten primary engineering challenges, their diagnosed root causes, and the concrete technical remediations implemented in the codebase.

Identified Challenge	Discovered Root Cause	Architectural Remediation Implemented	Evidence Citation
Identical Canned Prediction & Mock Fallback (P0 Defect)	Health worker testing showed every symptom input produced an identical 88.5% confidence prediction for 'Infectious Gastroenteritis'.	Traced to in-process import failure inside container falling back to legacy mock. Rewrote UI to route strictly through backend API `POST /api/v1/consultation/triage`; eliminated all mock fallbacks; enforced fail-closed HTTP 503 MODEL_UNAVAILABLE on ML outage.	docs/evidence/p0_root_cause.txt, p0_docker_tests.txt
Massive Dataset Duplication & Evaluative Leakage	Automated data provenance analysis on raw Kaggle dataset (4,920 rows) revealed identical symptom vectors across splits.	Discovered 4,616 exact duplicate rows (93.8% duplication). Dropped duplicates to 304 unique ground-truth rows (~7.4 per class); instituted 5x3 Repeated Stratified K-Fold CV with in-fold symptom dropout to eliminate evaluation leakage.	docs/evidence/data_provenance.txt, MODEL_CARD.md
Hardcoded Cryptographic Fallback Keys in Source Code	Gitleaks audit and static analysis detected fallback encryption keys embedded in `src/utils/cryptoUtils.js` and `dataEncryption.js`.	Removed all hardcoded fallback keys from source repository; updated gitleaks baseline with written rationales; implemented production startup guard `encryption_key_exit.test.js` verifying process exits with code 1 if ENCRYPTION_KEY is unset.	docs/evidence/gitleaks_report.txt, Backend/tests/encryption_key_exit.test.js
Unevidenced Accuracy Figures in Initial Report Draft	Fact-checking audit found numbers (99.67%, 95.39%, 83.22%, F1 0.9512, ECE 0.0384, Brier 0.0121) lacked supporting evidence files.	Completely excised unevidenced draft figures from report text; reconciled all performance claims with traceable nested CV benchmarks in `model_benchmark.md` and `tuned_tree_benchmark.md`.	docs/evidence/model_benchmark.md, tuned_tree_benchmark.md, COMPLETION_LEDGER.md
Unrealistic Load Test Stack & Concurrency Bottlenecks	Early load test issued requests against production rate limiters, recording 429s as normal behavior while list endpoint had 2.4s p99 latency.	Created dedicated `docker-compose.test.yml` with test-only limits (`ALLOW_LOAD_TEST=true`, `LLM_ENABLED=false`); pre-created pending consultations; optimized worker list with server-side pagination and projection (60.5% p50 latency speedup).	docs/evidence/load_test_testmode_results.txt, load_test_profiling_notes.md

Garbled LLM Output from Substring Confidence Rewriting	Naive in-place regex replacement of confidence terms ('certain', 'high') mangled valid clinical words like 'certainly' or 'ascertain'.	Replaced regex replacement with whole-phrase boundary matching; on detection of banned confidence terms, triggers one LLM regeneration; if still failing, falls back cleanly to deterministic clinical template.	scripts/test_confidence_word_filter.py, docs/evidence/p2_guardrails_tests.txt
Gemini Model Deprecation & API Quota Exhaustion	Gemini 1.5 Flash returned 404 model not found; free-tier API quota exhausted after 20 daily calls returning HTTP 429.	Implemented dynamic ordered cascade `LLM_MODELS = ['gemini-2.5-flash', 'gemini-3.5-flash', 'gemini-3-flash-preview']`; added 10s per-attempt timeout and 20s overall cap; engaged deterministic safe template on exhaustion.	docs/evidence/gemini_live_run.txt, scripts/test_llm_timeouts_and_429.py
Container Vulnerabilities in Base Docker Images	Trivy container security scans reported Critical and High vulnerabilities in Python and Node.js images.	Implemented multi-stage Docker build for backend stripping npm CLI; uninstalled pip and ensurepip from runtime stages of Python images; upgraded critical packages; reduced fixable vulnerabilities to 0 with `--ignore-unfixed`.	docs/evidence/trivy_all_images_scan.txt, trivy_all_images_scan_notes.md
Worker List Endpoint Latency Degradation	Load test at 50 concurrent users showed `GET /consultation/my` taking 7.0s p99 due to full collection scan and JSON transfer of all patient records.	Added server-side pagination (`page`, `limit`, default 20) and lean Mongoose projection excluding heavy internal audit vectors; reduced C=1 p50 latency from 193 ms to 76.2 ms (60.5% speedup) and p99 from 660.6 ms to 152.0 ms (77.0% speedup).	docs/evidence/load_test_profiling_notes.md, load_test_testmode_results.txt
Silent Auth Failure / Session Expiry in Streamlit UI	When JWT access token expired after 15 minutes, UI failed silently or crashed without clear guidance to health worker.	Added explicit 401 interceptor in `ui/utils/api_client.py` displaying prominent warning: 'Your session has expired. Please log in again to continue.'; sanitized state without data loss.	ui/utils/api_client.py, docs/evidence/verify_ui.txt

Table A.1: Engineering Challenges, Root Causes, and Architectural Remediation Fixes

Appendix B: Reproduction Commands & Verification Playbook

To ensure total scientific reproducibility, this appendix provides the complete sequence of command-line instructions required to launch the ArogyaMitra microservice cluster, execute all 321 automated test suites, run the 20 golden clinical test cases, execute the Playwright UI automation, and run the security scanners from a clean development environment.

Step 1: Clone Repository & Build Docker Stack

```
git checkout remediation
docker compose -f docker-compose.yml build
docker compose -f docker-compose.yml up -d
powershell -File scripts\health_check.ps1
```

Step 2: Seed Clean Test Environment

```
powershell -File scripts\seed_docker.ps1
# Verifies database initialized with 6 healthy services and clean test users
```

Step 3: Execute Backend API Unit & Integration Suite (231 Tests)

```
docker compose exec -T backend npm test -- --coverage
# Output recorded in docs/evidence/verify_backend.txt
```

Step 4: Execute Machine Learning Inference Suite (55 Tests)

```
docker compose exec -T ml pytest -v
# Output recorded in docs/evidence/verify_ml.txt
```

Step 5: Execute Guarded LLM Narrative Suite (26 Tests)

```
docker compose exec -T graph pytest -v
# Output recorded in docs/evidence/verify_graph.txt
```

Step 6: Execute Frontend UI Test Suite (9 Tests)

```
docker compose exec -T ui pytest -v
# Output recorded in docs/evidence/verify_ui.txt
```

Step 7: Run Golden Clinical Validation Set (20 Vignettes)

```
python scripts\test_p0_golden_set.py
# Output recorded in docs/evidence/p0_golden_tests.txt
```

Step 8: Run Playwright Browser Automation Suite (11 Journeys)

```
node scripts\test_p4_playwright_suite.mjs
# Output recorded in docs/evidence/p4_playwright_suite.txt
```

Step 9: Execute Multi-Tier Load Test (2,400 Requests across C=1, 10, 50)

```
docker compose -f docker-compose.test.yml up -d
python scripts\run_p5_load_test_testmode.py
# Output recorded in docs/evidence/load_test_testmode_results.txt
```

Step 10: Run Container Vulnerability Auditing with Trivy

```
powershell -File scripts\run_trivy_scan_all.ps1
# Output recorded in docs/evidence/trivy_all_images_scan.txt
```

Appendix C: Complete Prototype Test Suite Inventory

Table C.1 catalogues the 321 automated prototype tests verifying system correctness, security guardrails, fail-closed behavior, and database integrity across all microservices.

Target Service	Test Suite File	Test Count	Functional Scope & Invariants Tested
Backend	tests/triage_flow.test.js	**18 Tests**	Tests full triage payload ingestion, Pydantic validation, and red-flag short-circuiting.
Backend	tests/redflag_screening.test.js	**14 Tests**	Verifies all 5 emergency rules, boundary combinations, and override guarantees.
Backend	tests/auth_argon2.test.js	**22 Tests**	Validates password hashing, salt uniqueness, timing resistance, and role verification.
Backend	tests/aes_encryption.test.js	**16 Tests**	Verifies AES-256-GCM encryption at rest, unique IV generation, and deciphering.
Backend	tests/encryption_key_exit.test.js	**8 Tests**	Production guard verifying backend terminates immediately if encryption key is unset.
Backend	tests/concurrency_occ.test.js	**24 Tests**	Tests Mongoose `__v` optimistic locking and rejection of concurrent colliding updates.
Backend	tests/pagination_projection.test.js	**20 Tests**	Validates server-side pagination, limit bounds, bad input handling, and lean projection.
Backend	tests/api_security_headers.test.js	**28 Tests**	Verifies HSTS, X-Content-Type-Options, CSP, and CORS across endpoints.
Backend	tests/rate_limit.test.js	**16 Tests**	Tests IP rate limiters, 429 status generation, and test-mode limit relaxation.
Backend	tests/dpdp_consent.test.js	**14 Tests**	Tests patient consent logging, revocation requests, and data anonymization triggers.
Backend	tests/doctor_queue.test.js	**25 Tests**	Verifies doctor consultation retrieval, status filtering, and clinician response notes.
Backend	tests/fail_closed_ml.test.js	**26 Tests**	Verifies 503 MODEL_UNAVAILABLE error generation when ML service is down.
ML Engine	tests/test_model_inference.py	**15 Tests**	Tests BernoulliNB prediction, output shape, class rankings, and Softmax normalization.
ML Engine	tests/test_log_odds_attribution.py	**12 Tests**	Verifies exact additive log-odds formula and additivity to machine precision (<1e-10).

ML Engine	tests/test_vocabulary_validation.py	**10 Tests**	Tests 131 symptom vectorization, zero-padding, and invalid symptom rejection.
ML Engine	tests/test_missing_dropout.py	**18 Tests**	Verifies model stability under 0%, 30%, and 50% random symptom masking.
Graph LLM	tests/test_llm_timeouts.py	**8 Tests**	Verifies 10s per-attempt timeout and 20s hard deadline cap enforcement.
Graph LLM	tests/test_model_cascade.py	**6 Tests**	Tests ordered cascade from gemini-2.5-flash to fallback models on 429/404.
Graph LLM	tests/test_confidence_filter.py	**6 Tests**	Tests whole-phrase banned confidence detection and regeneration trigger.
Graph LLM	tests/test_template_fallback.py	**6 Tests**	Verifies deterministic template rendering when LLM is offline or times out.
Frontend UI	tests/test_ui_views.py	**5 Tests**	Validates Streamlit page rendering, role authentication, and session expiry warnings.
Frontend UI	tests/test_triage_form.py	**4 Tests**	Tests symptom selection state, payload formatting, and submission handling.

Table C.1: Detailed Inventory of 321 Prototype Automated Test Cases